# TrustSync build pack: phased prompts for Antigravity

Oct 12, 2026 · @irohansh

Fourteen phases, each with a BUILD prompt and a VERIFY prompt; nothing advances until the full regression suite and that phase's gate both pass.

## Start here

The agent never decides architecture: a frozen `docs/SPEC.md` does. You paste one small prompt at a time, and every phase ends with a test gate that also re-runs all earlier gates.

**The loop for every phase**

1. New Antigravity chat, paste the **Session-start prompt** (next section).
2. Paste the phase's **BUILD prompt**.
3. Run the gate yourself: `python tools/gates/run_gate.py N` (it runs gates 0 to N, plus `pytest -q`).
4. Open a **new chat** and paste the phase's **VERIFY prompt**. Fresh context means a skeptical reviewer instead of the same model grading its own work.
5. All green: `git tag phase-N-pass`. Any red: use the Debug prompt, at most 2 attempts, then roll back to the previous tag and split the BUILD prompt in half.

**Why this stops cascading failure**

- Interfaces are frozen in SPEC.md, so every module can be tested alone with synthetic data. Most gates need no camera or microphone.
- Gate N re-runs every earlier gate. A Phase 6 change that breaks Phase 2 is caught the same hour.
- One git tag per passing phase gives you a known-good rollback point.
- Tests that need real hardware carry `@pytest.mark.hardware` and run only when you ask for them; the rest run anywhere.

**Antigravity conventions used here** (checked against the current docs)

- Always-on rules live in `AGENTS.md` at the repo root (Antigravity also reads `GEMINI.md`). Each rules or workflow file is capped at 12,000 characters.

- Skills live in `.agents/skills/<name>/SKILL.md`; the older `.agent/` folder is still read. The agent loads a skill when its description matches the task.
- Workflows are markdown files in `.agents/workflows/` that you trigger as slash commands.

**Prompt style.** These are written for a fast, smaller model: one deliverable per prompt, exact file names, exact function signatures, an explicit DO-NOT list, and a fixed report-back format. If Antigravity offers Planning mode, use it for phases 5, 6, 9 and 10; Fast mode is fine elsewhere. Keep terminal commands on manual approval until Phase 3 passes.

**Three deliberate changes from your proposal**

- `sounddevice` replaces PyAudio for capture (installs without a compiler); librosa is not needed because the envelope is plain NumPy.
- The trust gate is built twice: a transparent rule-based gate first (Phase 6), then the scikit-learn classifier (Phase 8), so you can show an ablation.
- MediaPipe is pinned to a version that keeps the legacy `mp.solutions.face_mesh` API the spec uses.

| Phase | Deliverable | Gate: what must be true |
|---|---|---|
| 0 | Scaffold, rules, skills, SPEC, env | Imports work; camera and mic open; pytest collects |
| 1 | Types, config, synthetic-signal helpers | Config round-trips; type tests pass |
| 2 | Audio envelope, VAD, file and live sources | Tone-burst speech boundaries within 60 ms; live meter moves |
| 3 | Face tracker, mouth opening, stable IDs | Synthetic geometry tests; webcam IDs stable for 10 s |
| 4 | Head pose, gaze, addressee classifier | Known angles recovered within 3 degrees; look-away flips result |
| 5 | Lip-audio sync scorer and attribution | Lag recovered; static mouth scores 0; null 95th percentile below threshold |
| 6 | Pipeline, utterance aggregator, rule gate, HUD, offline CLI | 4 synthetic scenarios give expected decisions; 15+ FPS live |
| 7 | Recorder, validator, recording protocol | A 10 s test recording passes the validator |

| Phase | Deliverable | Gate: what must be true |
|---|---|---|
| 8 | Metrics, threshold calibration, learned gate | results.json and figures generated; ablation table |
| 9 | FastAPI: MJPEG, WebSocket, control routes | TestClient tests; WebSocket schema valid |
| 10 | React dashboard on scripted mock data | Build and typecheck pass; browser check at 2 sizes, 0 console errors |
| 11 | UI wired to API, replay mode, results page | Live and replay work end to end in the browser |
| 12 | Quality score, fuzz tests, benchmark | Black, noise and occluded frames never ACCEPT; FPS and latency table |
| 13 | README, report assets, demo script | Clean clone installs; full regression green |

Sources: Antigravity skills docs, Authoring Antigravity Skills codelab.

## Universal prompts

Five prompts you reuse in every phase. Replace `<N>` and `<PATH>` before pasting.

**U1. Session-start prompt** (first message of every new chat)

```
You are working on the TrustSync project. Before doing anything:
1. Read AGENTS.md, docs/SPEC.md and PROGRESS.md in full.
2. Run: git status and git tag --list 'phase-*'
3. Reply with exactly three lines:
   (a) the last passed phase, (b) the phase I am about to ask for, (c) any
file in the repo that contradicts docs/SPEC.md (or 'none').
Do not write or change any code in this message.
```

**U2. Save-a-file-verbatim prompt** (used in Phase 0 for the files in the setup packs)

```
Save the text between the lines START FILE and END FILE as the file <PATH>,
byte for byte.
Do not fix, reformat, summarise, translate or improve it. Create parent
folders if needed.
Then print the number of lines in the saved file.
START FILE
<paste the file content here>
END FILE
```

### U3. **Debug prompt** (when a test or gate fails)

```
A test or gate failed. Rules: do not edit any test file, do not edit
docs/SPEC.md, do not add try/except to hide the error.
1. Re-run the failing command and paste the last 40 lines of its output.
2. State the root cause in one sentence, naming the file and function.
3. Make the smallest change in source code that fixes that root cause.
4. Re-run the failing command, then run: python tools/gates/run_gate.py <N>
5. Report: root cause, files changed, gate result.
If it still fails after this one attempt, stop and write STUCK followed by
what you observed. Do not try a different approach on your own.
```

### U4. **Rollback prompt** (after 2 failed debug attempts)

```
Roll back to the last known-good state. Run: git stash -u
Then run: git checkout -b retry-phase-<N> phase-<N-1>-pass
Confirm with: git log --oneline -3 and python tools/gates/run_gate.py <N-1>
Do not start any new work. Report the output of those two commands.
```

### U5. **Standard VERIFY prompt** (new chat; each phase adds its own checks at the end)

```
You are a skeptical reviewer, not the author of this code. You may add tests
and run commands. You may NOT edit files under src/ and you may NOT change a
test to make it pass.
1. Read docs/SPEC.md and the files created in Phase <N>.
2. Run: python tools/gates/run_gate.py <N>   and   pytest -q -m "not
hardware"
3. Perform the phase-specific checks listed below, one at a time, printing
the exact command and its real output.
4. Look for: numbers hard-coded instead of read from config.yaml, a function
whose signature differs from docs/SPEC.md, tests that assert nothing, bare
except, print() instead of logging, files outside the tree in SPEC.md.
5. Final answer: a table with columns check, PASS or FAIL, evidence. Then a
list of defects as file:line. Do not fix anything.
Phase-specific checks:
<paste the checks from the phase here>
```

After a green VERIFY, you commit and tag by hand:

```
git add -A
git commit -m "phase N"
git tag phase-N-pass
```

`PROGRESS.md` is kept by the agent through the `/status` workflow (Setup pack B). It has one line per phase: number, status, date, gate output summary.

## Setup pack A: rules and the frozen spec

Two files. In Phase 0 you save them verbatim with prompt U2. `AGENTS.md` is read by Antigravity on every task; `docs/SPEC.md` is what every phase prompt points at. Do not let the agent edit either one later.

**File 1: `AGENTS.md`** (about 3,000 characters, well under the 12,000 limit)

```
# TrustSync agent rules (always on)

Project: a real-time perception module for a social robot. A camera and a
microphone feed a pipeline that decides whether a spoken command was (1)
spoken live, (2) spoken by which visible face, (3) directed at the robot.
Output is ACCEPT or a REJECT_* reason.

## Source of truth
- docs/SPEC.md is frozen. Never change it. Never rename or re-sign a file,
```

```
class, function, field or config key it defines. If SPEC.md looks wrong, STOP
and ask the human.
- Build only what the current prompt asks for. No extra features. No
refactors of earlier phases.

## Working rules
1. Python 3.11. Type hints on every function. Document numpy array shapes in
docstrings.
2. Every threshold and constant comes from config.yaml through
src/trustsync/config.py. No magic numbers in code.
3. No global mutable state. No sleep() in library code. Use logging, never
print(), outside tools/.
4. Time: every timestamp is float seconds from time.monotonic(). Video and
audio share this clock.
5. Library code must run without a camera or microphone. Hardware is touched
only in src/trustsync/io/ and tools/.
6. Tests: pytest, seeded randomness (np.random.default_rng(1234)), no
network, no single test over 5 s. Hardware tests carry @pytest.mark.hardware.
7. Never edit a test to make it pass. Never delete a failing test. Never hide
an error with try/except.
8. Never install a package that is not in requirements.txt without asking.
9. Run tests: pytest -q -m "not hardware"   Run gates: python
tools/gates/run_gate.py N
10. Tests never read live values from config.yaml (calibration edits it).
They use the spec_cfg fixture in tests/conftest.py, which loads
tests/fixtures/config_spec_defaults.yaml. Never edit that fixture.

## Report-back format (end of every task)
FILES CREATED OR CHANGED: list
COMMANDS RUN: each with PASS or FAIL
GATE RESULT: the final table
OPEN PROBLEMS: list, or none
Never claim a command passed unless you ran it and saw its output.

## Things that fail silently in this project
- OpenCV frames are BGR; MediaPipe needs RGB.
- MediaPipe landmarks are normalized 0..1; multiply x by width and y by
height for pixels.
- Audio is 16000 Hz, mono, float32 in [-1, 1].
- Pearson correlation of a constant signal is NaN; guard std < 1e-8.
- Windows: use pathlib.Path, never hard-coded slashes.
```

**File 2: `docs/SPEC.md`** (the contract; every phase prompt refers to its section numbers)

```
# TrustSync SPEC v1 (FROZEN)

## 1. Stack
Python 3.11. requirements.txt (install mediapipe FIRST; it brings its own
opencv-contrib-python; never install opencv-python):
mediapipe==0.10.14
numpy==1.26.4
scipy==1.11.4
sounddevice==0.4.7
soundfile==0.12.1
PyYAML==6.0.1
scikit-learn==1.4.2
joblib==1.4.2
pandas==2.2.2
matplotlib==3.8.4
fastapi==0.111.0
uvicorn[standard]==0.30.1
httpx==0.27.0
pytest==8.2.2
pyproject.toml: package 'trustsync' in src/, pytest pythonpath=['src'],
marker 'hardware'. Install with pip install -e .

## 2. Tree
trustsync/
  AGENTS.md PROGRESS.md README.md requirements.txt pyproject.toml config.yaml
.gitignore
  docs/SPEC.md
  .agents/rules .agents/skills .agents/workflows
  src/trustsync/
    __init__.py config.py types.py synth.py pipeline.py utterance.py
    audio/{__init__,envelope,sources}.py
    vision/{__init__,face_tracker,id_tracker,lip,quality,head_pose,gaze}.py
    addressee/{__init__,classifier}.py
    sync/{__init__,scorer}.py
    fusion/{__init__,gate,learned}.py
    io/{__init__,video_source,recorder}.py
    overlay/{__init__,hud}.py
    eval/{__init__,metrics,calibrate,run_eval,plots}.py
    server/{__init__,app,runtime}.py

tools/{run_offline,record_session,validate_session,capture_sample,calibrate_p
ose,benchmark,demo_live,audio_meter,make_demo_session,run_demo,make_report_as
sets,make_report_skeleton}.py
```

```
  tools/gates/{run_gate.py, gate_0.py ... gate_13.py}
  tests/conftest.py tests/helpers_world.py tests/helpers_eval.py
tests/test_*.py tests/fixtures/ (config_spec_defaults.yaml and your face
jpgs)
  docs/ARCHITECTURE.md docs/RECORDING.md   report/ (generated in Phase 13)
out/ (scratch, git-ignored)
  data/sessions/<session_id>/{video.mp4,audio.wav,frame_times.csv,meta.json}
  models/  reports/figures/  reports/results.json  web/

## 3. config.yaml (exact keys; write in normal block style; * = re-calibrated
in Phase 8)
video: fps 30, width 640, height 480, camera_index 0, max_faces 4
audio: sample_rate 16000, env_hz 100, device null, latency_s 0.0, ring_s 10.0
vad: calib_s 1.0, on_factor 3.0, min_abs 0.004, hangover_s 0.30, adapt 0.005
utterance: min_s 0.40, max_s 4.0, pad_s 0.25
tracker: max_dist 0.15, max_missed 15
lip: buffer_s 6.0
sync: window_s 1.5, min_window_s 0.6, max_lag_s 0.20, min_lip_std 0.004*,
env_eps 0.0001
quality: min_face_px 80, min_brightness 40, max_edge_frac 0.05
addressee: max_yaw_deg 25*, max_pitch_deg 20*, yaw_offset_deg 0.0,
pitch_offset_deg 0.0, gaze_weight 0.3, gaze_max_off 0.6, use_gaze true
gate: mode rule, min_sync 0.35*, min_margin 0.10*, min_directed 0.5*,
min_quality 0.5*, model_path models/gate.joblib
server: host 127.0.0.1, port 8000, ws_hz 20, jpeg_quality 80
config.py: one frozen dataclass per section (VideoCfg, AudioCfg, VadCfg,
UtteranceCfg, TrackerCfg, LipCfg, SyncCfg, QualityCfg, AddresseeCfg, GateCfg,
ServerCfg) whose field names equal the keys, and Config holding them under
the section names. load_config(path: Path|None=None) -> Config (default:
repo-root config.yaml); save_config(cfg, path); Config.to_dict().

## 4. Types (src/trustsync/types.py, all dataclasses)
DECISIONS =
('IDLE','LISTENING','ACCEPT','REJECT_SPOOF','REJECT_AMBIGUOUS','REJECT_NOT_AD
DRESSED','REJECT_LOW_QUALITY','REJECT_NO_FACE')
FaceObs(frozen): track_id:int, bbox:tuple[float,float,float,float]
(x1,y1,x2,y2 normalized 0..1), pts:np.ndarray (478,3) normalized x,y plus
relative z, mouth_open:float, quality:float
FaceScores: track_id:int, bbox, sync:float, lip_std:float, best_lag_s:float,
yaw:float, pitch:float, roll:float, gaze_off:float, directed:float,
quality:float
SyncResult: score:float, lag_s:float, lip_std:float, valid:bool,
r_open:float, r_vel:float
```

```
TrackSummary: track_id:int, sync:float, lip_std:float, directed:float,
quality:float, yaw:float, pitch:float
UtteranceSummary: t_start:float, t_end:float, duration_s:float,
tracks:list[TrackSummary]
GateDecision: decision:str, reason:str, speaker_id:int|None, liveness:float,
attribution_conf:float, addressee:float
FrameResult: t:float, frame_idx:int, fps:float, faces:list[FaceScores],
speech_active:bool, env:float, noise_floor:float, speaker_id:int|None,
attribution_conf:float, liveness:float, addressee:float, decision:str,
reason:str, utterance_final:bool
frame_result_to_dict(r: FrameResult) -> dict: JSON-safe (python floats
rounded to 4 decimals, nan becomes null, tuples become lists).

## 5. Modules, signatures and exact algorithms
### audio/envelope.py
compute_envelope(x: np.ndarray, sr: int, env_hz: int) -> np.ndarray: float32
RMS; hop = sr//env_hz; window = 2*hop centred on each hop; one value per hop,
length ceil(len(x)/hop); zero-pad the end.
class VAD(cfg: VadCfg, dt: float): update(env: float) -> bool; properties
noise_floor, calibrated. For the first calib_s seconds (count updates*dt)
noise_floor = mean(env) and it returns False. Afterwards on = env >
max(noise_floor*on_factor, min_abs); output stays True for hangover_s after
the last on sample; while output is False, noise_floor = (1-
adapt)*noise_floor + adapt*env.
segment_speech(env: np.ndarray, env_hz: float, cfg: VadCfg) ->
list[tuple[float,float]]: runs VAD with dt=1/env_hz; each segment is (time of
first on sample, time of last on sample); trailing hangover is trimmed.
### audio/sources.py
class AudioSource: envelope_at(times: np.ndarray) -> np.ndarray (linear
interpolation on the stored envelope timeline; 0.0 outside the covered
range); latest() -> tuple[float,float] (t, env); close().
NumpyAudioSource(env: np.ndarray, env_hz: float, t0: float) : env[i] is at
time t0 + i/env_hz.
FileAudioSource(wav_path: Path, t0: float, cfg: AudioCfg): soundfile read,
mix to mono, require sample rate == cfg.sample_rate (else ValueError),
compute_envelope, delegate to NumpyAudioSource.
LiveAudioSource(cfg: AudioCfg, record_to: Path|None=None):
sounddevice.InputStream(samplerate, channels=1, blocksize=sr//env_hz, dtype
float32, device cfg.device). Callback computes ONE envelope value (block RMS)
with timestamp time.monotonic() - cfg.latency_s - 0.5*block_s; keeps the last
ring_s seconds in a lock-protected deque. If record_to is set it also writes
raw samples to that WAV and exposes t0 (monotonic time of the first sample).
start(), close().
```

```
### vision
lip.mouth_open(pts, w, h) -> float: pixel coords (x*w, y*h); gap =
dist(landmark 13, landmark 14); iod = dist(landmark 33, landmark 263); return
gap/iod, or 0.0 if iod < 1e-6.
id_tracker.IdTracker(max_dist: float, max_missed: int).assign(centroids:
np.ndarray (n,2) normalized) -> list[int]: Hungarian matching
(scipy.optimize.linear_sum_assignment) on euclidean distance to existing
tracks; a pair with distance > max_dist is rejected; unmatched detections get
new ids 0,1,2,... that are never reused; a track unmatched for more than
max_missed consecutive calls is deleted. Centroid = centre of the landmark
bounding box.
quality.face_quality(frame_bgr, bbox, pts, cfg: QualityCfg) -> float =
s_size*s_bright*s_edge. s_size = clip(face_height_px/min_face_px,0,1).
s_bright = clip(mean grayscale inside bbox / min_brightness,0,1). s_edge =
1.0 if the fraction of landmarks with x or y outside [0.02,0.98] is <=
max_edge_frac else 0.0.
face_tracker.FaceTracker(cfg: Config): process(frame_bgr) -> list[FaceObs];
close(). Uses mediapipe.solutions.face_mesh.FaceMesh(static_image_mode=False,
max_num_faces=cfg.video.max_faces, refine_landmarks=True,
min_detection_confidence=0.5, min_tracking_confidence=0.5) on the RGB frame.
Returns [] when no face.
head_pose.estimate_pose(pts, w, h) -> tuple[float,float,float] (yaw, pitch,
roll in degrees; NaN triple on failure). Landmarks [1,152,33,263,61,291] map
to this 3D model (x right, y down, z away from camera, so a frontal face
gives angles near 0): 1:(0,0,0) 152:(0,330,65) 33:(-225,-170,135) 263:
(225,-170,135) 61:(-150,150,125) 291:(150,150,125). Camera matrix f=w,
cx=w/2, cy=h/2, zero distortion, cv2.solvePnP with SOLVEPNP_ITERATIVE. R =
cv2.Rodrigues(rvec)[0]; pitch = atan2(R[2,1], R[2,2]); yaw = atan2(-R[2,0],
sqrt(R[2,1]**2+R[2,2]**2)); roll = atan2(R[1,0], R[0,0]); convert to degrees.
gaze.gaze_off(pts, w, h) -> float: right eye r1 = (x468 - x33)/(x133 - x33);
left eye r2 = (x473 - x362)/(x263 - x362); both are 0.5 when the iris is
centred. gaze_off = clip(mean(|r1-0.5|,|r2-0.5|)*2, 0, 1). Return 0.0 if a
denominator is below 1e-6.
### addressee/classifier.py
directed_score(yaw, pitch, gaze_off, cfg: AddresseeCfg) -> float: yc = yaw -
yaw_offset_deg; pc = pitch - pitch_offset_deg; d = max(|yc|/max_yaw_deg,
|pc|/max_pitch_deg); pose = 1.0 if d <= 1 else max(0.0, 2.0 - d); if
use_gaze: pose *= 1 - gaze_weight*min(1, gaze_off/gaze_max_off); any NaN
input gives 0.0.
### sync/scorer.py
sync_score(lip: np.ndarray, env: np.ndarray, fps: float, cfg: SyncCfg) ->
SyncResult. Both arrays have the same length N and are sampled on the same
uniform grid at fps.
```

```
 1. If N < round(min_window_s*fps): SyncResult(0,0,0,valid=False,0,0).
 2. k = round(fps); lip_d = lip - scipy.ndimage.uniform_filter1d(lip, k,
mode='nearest'); lip_std = std(lip_d). If lip_std < min_lip_std: return score
0.0, valid=True (static mouth).
 3. Features: f_open = lip_d; f_vel =
uniform_filter1d(abs(gradient(lip_d))*fps, 3, mode='nearest'). Audio: e =
log(env + env_eps).
 4. K = round(max_lag_s*fps). For each lag j in -K..K: if j >= 0 pair a =
f[:N-j] with b = e[j:], else a = f[-j:] with b = e[:N+j]. r = Pearson(a, b)
computed on that overlap (0.0 if either std < 1e-8). Positive lag means the
audio is later than the lip motion.
 5. For each feature take the maximum r over lags and its lag. score = max(0,
r_open_best, r_vel_best); lag_s = lag of the winning feature / fps. Fill
r_open, r_vel with the per-feature maxima.
attribute(scores: dict[int,float]) -> tuple[int|None, float]: (id of the
highest score, highest minus second highest where second = 0.0 if only one
id); empty dict gives (None, 0.0).
### fusion/gate.py
class Gate: decide(summary: UtteranceSummary) -> GateDecision. RuleGate(cfg:
Config) follows section 6. LearnedGate is defined in Phase 8 and then frozen.
### pipeline.py and utterance.py
TrustSyncPipeline(cfg: Config, audio: AudioSource, gate: Gate|None=None,
tracker=None): process(frame_bgr: np.ndarray, t: float) -> FrameResult.
'tracker' is injectable (anything with process(frame_bgr)->list[FaceObs]);
default FaceTracker. Per call:
 1. faces = tracker.process(frame). 2. env_now = audio.envelope_at([t])[0];
speech = vad.update(env_now), VAD dt = 1/video.fps.
 3. For every face: append (t, mouth_open) to that track's buffer (keep
lip.buffer_s seconds); yaw,pitch,roll = estimate_pose; gaze = gaze_off if
use_gaze else 0; directed = directed_score; running sync = sync_score on the
last window_s (grid at video.fps, lip via np.interp, env via
audio.envelope_at); if the buffer covers less than min_window_s the running
sync is 0. Running sync is for display only.
 4. Utterance state machine (UtteranceTracker): starts when speech becomes
True; while active the decision is LISTENING; ends when speech becomes False
or elapsed >= utterance.max_s. If duration < utterance.min_s: discard
(decision IDLE). Otherwise build UtteranceSummary: per track, sync =
sync_score over the grid [t_start - pad_s, t_end + pad_s] at video.fps (a
track whose buffer covers < 80 percent of that span gets sync 0 and lip_std
0); directed, quality, yaw, pitch = median over the utterance frames in which
the track was visible. gate.decide(summary) gives the final decision; that
frame has utterance_final=True. The next frame returns to IDLE.
 5. fps = exponential moving average of 1/(time between process calls), alpha
```

```
0.1.
### io, overlay, synth
io/video_source.py: read() -> tuple[bool, np.ndarray|None, float] (ok, frame,
t). WebcamSource(cfg) (cv2.VideoCapture; t = time.monotonic() right after
read). FileVideoSource(video_path, frame_times_csv) (t from csv column t).
overlay/hud.py: draw_hud(frame_bgr, result: FrameResult, env_hist:
list[float]) -> np.ndarray (returns a copy).
synth.py (test helpers, no hardware): make_env(duration_s, env_hz, bursts:
list[tuple[float,float]], rng) -> np.ndarray; make_lip(env, env_hz, fps,
lag_s, gain, noise, rng) -> np.ndarray; make_static_lip(duration_s, fps,
noise, rng) -> np.ndarray; make_pts(yaw=0, pitch=0, roll=0, w=640, h=480,
mouth_gap_ratio=0.05, iris_ratio=0.5, center=(0.5,0.5), scale=1.0) ->
np.ndarray (478,3); class FakeFaceTracker(script) (script: list of
list[FaceObs], one entry per frame).

## 6. RuleGate decision order
 1. no tracks: REJECT_NO_FACE.
 2. max quality over tracks < min_quality: REJECT_LOW_QUALITY.
 3. best = track with the highest sync. If best.sync < min_sync or
best.lip_std < sync.min_lip_std: REJECT_SPOOF.
 4. two or more tracks and best.sync - second.sync < min_margin:
REJECT_AMBIGUOUS.
 5. best.quality < min_quality: REJECT_LOW_QUALITY.
 6. best.directed < min_directed: REJECT_NOT_ADDRESSED.
 7. otherwise ACCEPT with speaker_id = best.track_id.
 liveness = best.sync; attribution_conf = best.sync - second.sync (second = 0
if one track); addressee = best.directed. reason is a short human sentence
naming the numbers.

## 7. Session files and labels
meta.json keys: id, scenario, subjects (list of str), speaker_side
('left'|'right'|'center'|'none'), addressed (bool), expected
('ACCEPT'|'REJECT_SPOOF'|'REJECT_NOT_ADDRESSED'|'REJECT_ANY'|'NONE'),
lighting ('normal'|'dim'), occlusion (bool), spoof_device (null|'phone'),
audio_t0 (float, monotonic), video_fps (float), notes (str).
Scenarios and expected: live_single ACCEPT; replay_spoof REJECT_SPOOF;
multi_one_speaks ACCEPT (speaker_side set); not_addressed
REJECT_NOT_ADDRESSED; occlusion_lowlight REJECT_ANY; silence NONE (no
utterance may be produced).
frame_times.csv: header frame_idx,t. A track's side = left if bbox centre x <
0.4, right if > 0.6, else center.

## 8. HTTP API (Phase 9)
```

```
GET /api/health -> {ok, running, source}
GET /api/config -> config dict. POST /api/config {section,key,value} ->
updated dict (allowed only for sections gate and addressee, and
sync.min_lip_std).
GET /api/sessions -> [{id, scenario, expected}] from data/sessions.
POST /api/source {type:'webcam'|'session', id?} -> {ok}. POST /api/stop ->
{ok}.
GET /api/results -> reports/results.json, or 404.
GET /stream.mjpg -> multipart/x-mixed-replace, boundary=frame, raw camera
JPEG without overlay.
WS /ws -> first message {type:'hello', config, decisions}; then
{type:'frame', data: frame_result_to_dict} at server.ws_hz; on
utterance_final also {type:'utterance', data: same dict}; and {type:'status',
data:{running, source, error}} whenever the runtime starts, stops or fails
(source is 'webcam', 'session:<id>' or null; error is a string or null).

## 9. UI contract (Phases 10-11)
web/: Vite + React 18 + TypeScript + Tailwind 3.4 + lucide-react + recharts +
framer-motion, hand-written components (no shadcn CLI). web/src/types.ts
mirrors section 4. Mock mode is the URL query ?mock=1.
```

## Setup pack B: skills and workflows

Four skills and three slash workflows, all saved in Phase 0 with prompt U2. Skills load on their own when the task matches the description; workflows run when you type the slash command. The `modern-ui-design` skill is what makes the dashboard look current, so it carries the full design tokens. If you also have Anthropic's public `frontend-design` skill (same SKILL.md format), you can copy it into `.agents/skills/` as well, but it is optional.

**`.agents/skills/phase-gate/SKILL.md`**

```
---
name: phase-gate
description: Use when finishing, verifying or starting any TrustSync phase,
or when asked to run a gate, check for regressions, or report phase status.
---
# Phase gate procedure
1. python tools/gates/run_gate.py N runs gate_0.py through gate_N.py in order
and stops at the first failure. Each gate_K.py prints a table with columns
CHECK | RESULT | DETAIL and exits 0 only if every row is PASS.
2. Always also run: pytest -q -m "not hardware"
3. Hardware rows (camera, microphone, live FPS) print SKIP unless the script
is run with --hardware. A phase is tag-ready only after the human has run it
once with --hardware and every row is PASS.
4. Never say a phase is done unless run_gate.py N exited 0. Paste the final
table in your answer.
5. After a pass, append one line to PROGRESS.md: phase N | PASS | date | one-
line summary.
6. Later phases may ADD checks to earlier gate files; never weaken, skip or
delete an existing check.
```

**`.agents/skills/synthetic-signal-testing/SKILL.md`**

```
---
name: synthetic-signal-testing
description: Use when writing or reviewing tests for audio envelopes, VAD,
lip signals, sync scoring, head pose or any numeric code in TrustSync. Build
deterministic synthetic inputs instead of needing a camera or microphone.
---
# Rules for numeric tests
- Every test creates rng = np.random.default_rng(1234). Never use unseeded
randomness.
- Build inputs with the helpers in src/trustsync/synth.py.
- Each numeric assertion states its tolerance and the reason in a comment
(example: within 2 frames = 67 ms at 30 fps).
- Always write three cases: positive (must detect), negative (must not
detect), degenerate (empty, constant, NaN, too short).
- Statistical claims use at least 200 seeds and assert on percentiles, never
on a single draw.
- Assert a margin around a threshold, not just the threshold itself.
- Never loosen a tolerance to make a test pass. Fix the code, or report
STUCK.
- Known bug sources to test on purpose: lag sign, BGR vs RGB, normalized vs
pixel coordinates, seconds vs milliseconds, division by zero, off-by-one at
window edges.
```

**`.agents/skills/modern-ui-design/SKILL.md`**

```
---
name: modern-ui-design
description: Use for any work inside web/: building or changing the TrustSync
dashboard, components, styling, charts, layout or animation. Contains the
design tokens, layout grid, component rules and the visual verification
checklist.
---
# TrustSync UI design system
Mood: calm control room. Dark, high contrast, one colour per meaning,
generous spacing, numbers in a mono font.

## Tokens (CSS variables in web/src/index.css, mapped in tailwind.config.js)
--bg #0A0E14; --bg-elev #0F1520; --card rgba(255,255,255,0.04); --card-border
rgba(255,255,255,0.08); --text #E6EDF3; --muted #8B98A9; --accept #2DD4BF; --
reject #F87171; --listen #FBBF24; --info #A78BFA; --idle #64748B.
Radius: cards 16px, chips 999px, inputs 10px. Spacing scale 4 8 12 16 24 32.
Shadow 0 8px 30px rgba(0,0,0,.35).
```

```
Card: backdrop-blur-xl, 1px border var(--card-border), a soft radial glow in
the card's accent colour at 6 percent opacity in the top-left corner.
Fonts: Inter Variable for text and JetBrains Mono for numbers, installed from
@fontsource-variable/inter and @fontsource/jetbrains-mono. No CDN links: the
demo must work offline.

## Layout
Shell: 72px left rail (lucide icons with tooltips), top bar with a connection
chip and a source selector. Main area is a 12-column grid with 24px gaps.
Live page: video hero card 8 columns; right column 4 columns holding the
Decision card and the three Trust gauges; below, Sync timeline 8 columns and
Event log 4 columns.
Below 768px: the rail becomes a bottom tab bar and everything stacks in one
column with the video first.

## Rules
- Colour is semantic only: teal accept, red reject, amber listening, slate
idle. Never decorative.
- Every decision change animates for 200 ms (ease-out, scale and fade,
framer-motion). Respect prefers-reduced-motion.
- Every data component has three states: loading skeleton, empty state with a
hint, error state with a retry button.
- Text contrast on cards at least 4.5:1. Focus ring 2px in the accent colour.
Every icon-only button has an aria-label. Decision changes are announced in
an aria-live=polite region.
- Numbers use tabular-nums with fixed decimals: scores 2 dp, angles 0 dp with
a degree sign, fps 0 dp.
- Charts: horizontal gridlines only at 6 percent opacity, 2px lines, the
threshold as a dashed line labelled with its value, dark glass tooltip.
- No lorem ipsum, no emoji as icons (use lucide-react), no default blue
links.

## Verification (after every UI task)
1. npm run typecheck and npm run build must both pass.
2. Open the app with the browser tool at 1440x900 and at 390x844 and take a
screenshot of each.
3. Confirm: no horizontal scroll, no clipped text, no overlapping cards, 0
console errors, and ?mock=1 shows all of IDLE, LISTENING, ACCEPT and REJECT
states.
4. Describe what you saw in 3 bullets. Never claim a visual is fine without
having opened it.
```

**`.agents/skills/debug-root-cause/SKILL.md`**

```
---
name: debug-root-cause
description: Use when a test, gate, build or run fails in TrustSync, before
changing any code.
---
# Debug protocol
1. Reproduce: re-run the failing command and keep the last 40 lines of
output.
2. Localize: name the file and function where the first wrong value appears.
Print intermediate values; do not guess.
3. One hypothesis at a time. Change one thing, re-run, record the result.
4. Fix the cause in src/, never in the test, never by loosening a tolerance,
never by catching the exception.
5. After the fix, run the whole gate, not only the failing test.
6. After two failed attempts write STUCK, list what you ruled out, and stop.
```

**`.agents/workflows/gate.md`**

```
---
description: Run the cumulative gate for a phase and show the result table
---
1. Ask which phase number N, unless the user already gave it.
2. Run: python tools/gates/run_gate.py N
3. Run: pytest -q -m "not hardware"
4. Paste both final tables. If anything is FAIL, stop and list the failing
rows. Do not fix anything.
```

**`.agents/workflows/status.md`**

```
---
description: Refresh PROGRESS.md from git tags and the last gate run
---
1. Run: git tag --list 'phase-*' and git log --oneline -5
2. Rewrite PROGRESS.md as a table: phase | status (PASS or TODO) | date |
note. A phase is PASS only if its tag phase-N-pass exists.
3. Print the table.
```

**`.agents/workflows/rollback.md`**

```
---
description: Return to the last passing phase after failed attempts
---
1. Ask which phase N failed.
2. Run: git stash -u
3. Run: git checkout -b retry-phase-N phase-(N-1)-pass
4. Run: python tools/gates/run_gate.py (N-1) and print the table.
5. Do not start any new work.
```

## Phase 0: environment, scaffold, gate framework

Goal: an empty but runnable project, the rules and skills installed, and a gate runner that every later phase plugs into.

**You do this by hand first (10 minutes)**

- Install Python 3.11, Node 20 LTS (needed from Phase 10), Git, and Chrome (Antigravity's browser tool uses it).
- Make an empty folder `trustsync`, run `git init` inside it, open it as the Antigravity workspace.
- Allow camera and microphone access for your terminal or IDE in the OS privacy settings.

**0-A. Install the rules, spec, skills and workflows** (one message, 9 files: `AGENTS.md`, `docs/SPEC.md`, the 4 `SKILL.md` files under `.agents/skills/<name>/`, the 3 workflow files under `.agents/workflows/`)

Use prompt U2, repeating the START FILE / END FILE block once per file with its own path, all in the same message. Then reply to the agent: `Run: git add -A and git commit -m 'phase 0 files'`.

**0-B. BUILD prompt**

```
Start: read AGENTS.md and docs/SPEC.md. Phase 0 task: project scaffold and
gate framework. Do the steps in order and stop after step 9.

1. Create the folder tree from SPEC section 2. Put an empty __init__.py in
every package folder under src/trustsync. Put an empty .gitkeep in
data/sessions, models, reports/figures, tests/fixtures and web.
2. Write requirements.txt exactly as SPEC section 1, one package per line,
mediapipe first.
3. Write pyproject.toml: [build-system] requires setuptools; [project] name
trustsync, version 0.1.0, requires-python >=3.10,<3.12; setuptools
packages.find where=['src']; [tool.pytest.ini_options] pythonpath=['src'],
testpaths=['tests'], markers=['hardware: needs a camera or microphone'].
4. Write .gitignore containing: .venv, __pycache__, *.pyc, .pytest_cache,
out/, web/node_modules, web/dist, data/sessions/*/ .
5. Create the virtual environment in .venv, activate it, then run in this
order: pip install --upgrade pip ; pip install mediapipe==0.10.14 ; pip
install -r requirements.txt ; pip install -e .   Tell me which OS shell you
used. If pip reports a conflict or error, paste the full error and STOP.
6. Create tools/gates/run_gate.py. Usage: python tools/gates/run_gate.py N [-
-hardware]. Behaviour: (a) first run the subprocess  python -m pytest -q -m
"not hardware"  and record one row 'pytest'; pytest exit code 5 (no tests
collected) counts as PASS only when N is 0, otherwise FAIL. (b) For K from 0
to N import tools/gates/gate_K.py (use importlib with the file path). A
missing file is a FAIL row 'gate_K missing'. Each gate file defines
run(hardware: bool) -> list[tuple[str, str, str]] of (check, result, detail)
where result is PASS, FAIL or SKIP. (c) Print one aligned table with columns
GATE | CHECK | RESULT | DETAIL. (d) Exit code 1 if any FAIL, else 0. (e) Last
line: 'GATE N: PASS' or 'GATE N: FAIL (x failures)', and if any row was SKIP
add ' [hardware checks pending]'.
7. Create tools/gates/gate_0.py with these checks: python version is 3.10 or
3.11; each of these imports succeeds: numpy, scipy, cv2, mediapipe,
sounddevice, soundfile, yaml, sklearn, joblib, pandas, matplotlib, fastapi,
uvicorn, httpx; 'import cv2' came from opencv-contrib-python and pip has no
distribution named opencv-python (use importlib.metadata);
mediapipe.solutions.face_mesh is accessible and mediapipe version is 0.10.14;
every FOLDER named in SPEC section 2 exists (folders only, because the module
files are created in later phases; skip data/sessions/<session_id>), and
these files exist: AGENTS.md, PROGRESS.md, README.md, requirements.txt,
pyproject.toml, docs/SPEC.md, tools/gates/run_gate.py, tools/gates/gate_0.py;
AGENTS.md is shorter than 12000 characters; each of the 4 skills has SKILL.md
whose frontmatter name equals its folder name; git repository exists.
Hardware checks (SKIP unless hardware is True): cv2.VideoCapture(0) opens and
returns 5 frames (detail: width x height and measured FPS); sounddevice has a
```

```
default input device and a 1 second recording at 16000 Hz has RMS greater
than 0 (detail: device name and RMS).
8. Create PROGRESS.md containing a markdown table header: phase | status |
date | note, and rows for phases 0 to 13 all TODO. Create README.md with the
title TrustSync and one sentence describing the project.
9. Run: python tools/gates/run_gate.py 0   and fix only scaffold problems.

DO NOT write any library code in src/trustsync beyond empty __init__.py
files. DO NOT change docs/SPEC.md or AGENTS.md.
Report back in the format from AGENTS.md.
```

**0-C. You run the hardware check**

```
python tools/gates/run_gate.py 0 --hardware
```

Both the camera and microphone rows must say PASS. If the camera row fails, try `camera_index` 1 in your own test, then set it in `config.yaml` during Phase 1. If the microphone RMS is 0, check the OS input device.

**0-D. VERIFY** (new chat; paste U5 with N=0 and these checks)

```
1. Run python tools/gates/run_gate.py 0 and paste the table.
2. Run pip list and show the lines containing mediapipe, opencv, numpy,
scipy. Confirm there is no line for opencv-python (contrib is fine).
3. Print the character count of AGENTS.md and the line count of docs/SPEC.md.
Confirm docs/SPEC.md first line is: # TrustSync SPEC v1 (FROZEN)
4. List every file under .agents/ with its size.
5. Open tools/gates/run_gate.py and confirm: it exits 1 on any FAIL, it
prints the SKIP note, and it does not swallow exceptions inside gate files (a
crash in a gate must show as a FAIL row, with the exception text).
6. Ask me (the human) to type /status in the Antigravity chat and to ask:
'what does the phase-gate skill say about hardware rows?'. Report whether the
skill content was used in the answer.
```

Tag: `git add -A`, `git commit -m "phase 0"`, `git tag phase-0-pass`.

# Phase 1: config, types, synthetic data

Goal: the typed skeleton and the synthetic-signal generators that make every later phase testable without hardware.

**1-A. BUILD prompt**

```
Start: read AGENTS.md and docs/SPEC.md (sections 3, 4 and the synth.py part
of section 5). Phase 1 task. Create exactly these files and nothing else.

1. config.yaml at the repo root: every section and key from SPEC section 3
with the listed defaults, block style, one comment line above each section.
Then copy it byte for byte to tests/fixtures/config_spec_defaults.yaml. That
copy is never edited afterwards: tests read it instead of the live
config.yaml, because Phases 4 and 8 change values in config.yaml.
2. src/trustsync/config.py: frozen dataclasses per section, Config,
load_config, save_config, Config.to_dict as in SPEC section 3. Default path =
the config.yaml found by walking up from this file. Unknown yaml key: raise
ValueError. Missing key: raise ValueError naming it. Also validate(cfg) which
raises ValueError if any of these hold: audio.sample_rate % audio.env_hz !=
0; sync.max_lag_s * video.fps < 1; sync.min_window_s > sync.window_s;
utterance.min_s > utterance.max_s. load_config calls validate.
3. src/trustsync/types.py: DECISIONS and every dataclass of SPEC section 4
with exactly those field names in that order, plus frame_result_to_dict.
4. src/trustsync/synth.py with these functions (signatures from SPEC section
5):
   make_env(duration_s, env_hz, bursts, rng): baseline 0.002 +
0.001*abs(rng.standard_normal(n)). Inside each (start_s, end_s) burst add a
floor of 0.03 plus a sequence of syllables: the first syllable centre is
start_s + 0.1 and each next centre is the previous one plus rng.uniform(0.12,
0.30) seconds, until end_s; each syllable is a raised-cosine bump 0.5*
(1+cos(pi*u)) for |u| <= 1 with u = (t - centre)/0.09, peak amplitude
0.1*rng.uniform(0.5, 1.0); finally add 0.005*abs(rng.standard_normal) noise.
The irregular syllable timing is required: a periodic modulation would make
the lag estimate in Phase 5 ambiguous. Return float32 of length
round(duration_s*env_hz).
   make_lip(env, env_hz, fps, lag_s, gain, noise, rng): positive lag_s means
the AUDIO IS LATER than the lip motion. Build the time grid t =
arange(round(len(env)/env_hz*fps))/fps; sample the envelope at t + lag_s with
np.interp (values outside the range use the edge value); smooth with a 3-tap
uniform filter; normalise by max(env, 1e-6); lip = 0.02 + gain*normalised +
noise*rng.standard_normal. Default gain in callers is 0.05.
   make_static_lip(duration_s, fps, noise, rng): 0.02 +
noise*rng.standard_normal(round(duration_s*fps)).
   make_pts(...): returns float64 array (478,3), normalized x,y, z = 0, all
rows not mentioned below stay 0. Steps: (a) keep the 3D model of SPEC section
5 head_pose as a private constant _POSE_MODEL (a dict landmark index ->
(x,y,z)). (b) R = Rz(roll) @ Ry(yaw) @ Rx(pitch) using degrees; rvec =
cv2.Rodrigues(R)[0]. (c) f = w; tz = 3000/scale; tvec = ((cx-0.5)*tz, (cy-
0.5)*h*tz/w, tz) where (cx,cy) = center; camera matrix [[f,0,w/2],[0,f,h/2],
```

```
[0,0,1]]; cv2.projectPoints for the 6 model points; divide pixels by (w,h) to
normalize and store at their landmark indices 1,152,33,263,61,291. (d) Pixel
positions: m = mean of landmarks 61 and 291; iod = distance between 33 and
263; gap = mouth_gap_ratio*iod; landmark 13 = m + (0,-gap/2); landmark 14 = m
+ (0,+gap/2). (e) Eyes: landmark 133 = p33 + 0.3*(p263 - p33); landmark 362 =
p263 - 0.3*(p263 - p33); landmark 468 = p33 + iris_ratio*(p133 - p33);
landmark 473 = p362 + iris_ratio*(p263 - p362).
   class FakeFaceTracker: __init__(script) where script is a list (one entry
per frame) of lists of FaceObs; process(frame_bgr) returns the next entry,
then [] after the end; close() does nothing.
5. Tests (pytest, seeded rng = np.random.default_rng(1234)):
   tests/conftest.py: a pytest fixture spec_cfg that returns
load_config(tests/fixtures/config_spec_defaults.yaml). tests/test_config.py:
loading that fixture equals every SPEC value; the live config.yaml has
exactly the same sections and keys as the fixture and passes validate(); save
then load round-trips; unknown key and missing key each raise ValueError;
each of the 4 validate rules is triggered by a bad config and raises
ValueError.
   tests/test_types.py: DECISIONS equals the 8 names of SPEC section 4 in
order; frame_result_to_dict turns NaN into None, numpy float32 into float,
and json.dumps of the result succeeds.
   tests/test_synth.py: make_env length is exact and mean inside a burst is
more than 10 times the mean outside; make_lip with noise 0 and lag 0 has
Pearson correlation above 0.9 with the envelope resampled to fps; with lag_s
= 0.1 and fps 30 the best lag found by your own brute-force cross-correlation
inside the test is 3 frames plus or minus 1; make_static_lip std is below
0.002 when noise = 0.001; make_pts returns shape (478,3) with all used values
in [0,1]; at yaw 0 the points 33 and 263 are symmetric around the centre
within 0.02; the ratio dist(13,14)/dist(33,263) in pixels equals
mouth_gap_ratio within 1e-6; FakeFaceTracker returns the scripted lists in
order and then [].
6. tools/gates/gate_1.py: one row per test file (run pytest on that file
through subprocess, PASS if exit 0) plus a row 'import trustsync.config,
trustsync.types, trustsync.synth' that imports in under 2 seconds.
7. Run: python tools/gates/run_gate.py 1   Fix only what fails.

DO NOT create any other module. DO NOT use cv2 in config.py or types.py.
Report back in the AGENTS.md format.
```

### 1-B. VERIFY checks (U5 with N=1)

```
1. Write a throwaway script (not committed) that loads config.yaml with
PyYAML and compares every key and value against SPEC section 3 as a hand-
typed dict. Report any difference.
2. Print make_env(6, 100, [(1.5,2.5)], rng).mean() for the burst window and
for the first second.
3. Plot nothing; instead print the first 10 values of make_lip with lag 0.1
and with lag 0, to prove they are shifted.
4. Confirm pts from make_pts(yaw=20) differ from make_pts(yaw=0) in landmarks
33 and 263 x-values by at least 0.01.
5. Grep src/trustsync for the strings 'print(' and 'except:' and report
matches.
```

## Phase 2: audio pipeline (envelope, VAD, sources)

Goal: turn audio, from a file or a live microphone, into a 100 Hz envelope and a clean speech-on/speech-off signal.

### 2-A. BUILD prompt

```
Start: read AGENTS.md and docs/SPEC.md (section 5, audio parts) and
src/trustsync/config.py. Phase 2 task. Create exactly:

1. src/trustsync/audio/envelope.py with compute_envelope, class VAD,
segment_speech exactly as in SPEC section 5. compute_envelope must be
vectorised (no Python loop over samples), raise ValueError if x is not 1-D,
return an empty float32 array for empty input. For hop i the window is the
2*hop samples centred on the middle of that hop; samples outside the signal
count as zero.
2. src/trustsync/audio/sources.py with AudioSource, NumpyAudioSource,
FileAudioSource, LiveAudioSource exactly as in SPEC section 5. envelope_at
uses np.interp over the timeline and returns 0.0 outside it (use left=0.0,
right=0.0). LiveAudioSource must be thread safe and must not import
sounddevice at module import time (import inside __init__) so that tests run
on machines without audio hardware.
3. tools/audio_meter.py (hardware tool): opens LiveAudioSource and the VAD,
prints once per 100 ms a line like  [#####     ] 0.031 SPEECH  or  silence,
for 15 seconds, then exits.
4. Tests with seeded rng:
   tests/test_envelope.py: a 1 kHz sine of amplitude 0.5, 2 s at 16000 Hz,
gives interior envelope values within 5 percent of 0.3536; all-zero input
gives values below 1e-9; length equals ceil(len/hop); empty input gives empty
output; a 2-D array raises ValueError.
   tests/test_vad.py: using synth.make_env(6, 100, [(1.5,2.5),(4.0,4.6)],
rng): segment_speech returns exactly 2 segments and each start and end is
within 0.06 s of the truth; loud signal from t=0 gives False for the whole
first calib_s; two bursts separated by a 0.2 s gap merge into one segment;
separated by 0.5 s stay two; a noise floor that slowly ramps from 0.003 to
0.006 over 10 s causes no speech detection; VAD.noise_floor never becomes
negative.
   tests/test_sources.py: NumpyAudioSource returns exact values at grid
times, linear midpoints between them, and 0.0 outside; latest() returns the
last (t, value); FileAudioSource (write a WAV into tmp_path with soundfile)
honours t0 (envelope_at(t0+1.5) near the burst amplitude); a WAV at 44100 Hz
raises ValueError; a stereo WAV is mixed to mono. A test marked
@pytest.mark.hardware: LiveAudioSource for 3 seconds produces timestamps that
strictly increase and rms values that are not all zero.
5. tools/gates/gate_2.py: rows for the three test files plus a hardware row:
when hardware is True print 'SPEAK: stay silent 2 s, then say one-two-three
twice' and record 8 s with LiveAudioSource and VAD; PASS if segment_speech
finds between 1 and 4 segments and the first starts after 1.0 s.
```

```
6. Run: python tools/gates/run_gate.py 2

DO NOT touch earlier phases' files except to fix a bug you can prove with a
failing test. Report back in the AGENTS.md format.
```

### 2-B. You run it with the microphone

```
python tools/audio_meter.py
python tools/gates/run_gate.py 2 --hardware
```

### 2-C. VERIFY checks (U5 with N=2)

```
1. Time compute_envelope on 60 s of random noise at 16000 Hz. It must take
under 0.5 s. Report the time.
2. Open envelope.py and confirm there is no Python for-loop over samples
inside compute_envelope.
3. Make a WAV with silence (1 s), a 440 Hz tone (1 s), silence (1 s) using
soundfile, run FileAudioSource + segment_speech, and print the segment. It
must be near (1.0, 2.0).
4. Check the VAD for the case where speech begins during the 1 s calibration:
confirm the code documents this behaviour in the docstring.
5. Confirm LiveAudioSource has no import of sounddevice at module top level.
```

## Phase 3: face tracking and lip opening

Goal: MediaPipe faces with stable IDs, a scale-free mouth-opening number, and a per-face quality score.

### 3-A. BUILD prompt

```
Start: read AGENTS.md, docs/SPEC.md (section 5, vision parts) and
src/trustsync/synth.py. Phase 3 task. Create exactly:

1. src/trustsync/vision/id_tracker.py (IdTracker), lip.py (mouth_open),
quality.py (face_quality), face_tracker.py (FaceTracker), all exactly as SPEC
section 5. FaceTracker imports mediapipe inside __init__, converts BGR to RGB
with cv2.cvtColor before processing, never modifies the input frame, builds
pts as a float64 (478,3) array from the landmark list, bbox from the min and
max of the landmark x and y clipped to [0,1], and returns [] when MediaPipe
finds no face. Quality uses face_quality. Provide FaceTracker.close().
2. tools/capture_sample.py: usage  python tools/capture_sample.py NAME  .
Opens the camera from config, shows a preview window with the text 'Look at
the camera. Saving in 3...2...1', saves the frame to tests/fixtures/NAME.jpg
and prints the file path and image size. Press q to abort.
3. tools/demo_live.py with argument --stage faces (choices will grow in later
phases): opens the webcam through cv2.VideoCapture, runs FaceTracker, draws
each bbox with the text 'id N  mouth 0.00' and the measured FPS in the top-
left corner; q quits.
4. Tests (seeded; use synth.make_pts for geometry):
   tests/test_id_tracker.py: one face with 0.005 jitter over 100 calls keeps
id 0; two faces moving apart keep their ids; a face missing for 10 calls
(less than max_missed) and then reappearing nearby keeps its id; missing for
20 calls gets a NEW id; two faces crossing paths with a step of 0.02 per call
keep their ids; ids are never reused; a detection farther than max_dist from
every track gets a new id; fuzz: 1000 calls with 0 to 4 random centroids
never raise, always return len == number of rows, and ids within one call are
unique.
   tests/test_lip.py: mouth_open equals the ratio used in make_pts within 1e-
6 for ratios 0, 0.02, 0.05, 0.2; same value for scale 0.5 and for w,h =
1280,720; returns 0.0 when landmarks 33 and 263 coincide.
   tests/test_quality.py: a bright 480x640 frame with a face of height 200 px
gives above 0.9; a frame with mean grey 10 gives below 0.3; a face of 40 px
height gives at most 0.5; more than 5 percent of landmarks outside
[0.02,0.98] gives exactly 0.0.
   tests/test_face_tracker.py: a black frame gives []; the input frame is
unchanged after process (compare a copy). If tests/fixtures/face_frontal.jpg
exists: calling process on it 3 times gives at least 1 face, pts shape
(478,3), bbox with x1<x2 and y1<y2, the same track_id on all 3 calls,
mouth_open between 0 and 0.3, quality at least 0.5. If
tests/fixtures/face_mouth_open.jpg also exists, its mouth_open must exceed
the frontal one by at least 0.10. If a fixture is missing the test calls
pytest.skip with the text 'run tools/capture_sample.py NAME'.
5. tools/gates/gate_3.py: one row per test file; a row 'fixtures present'
```

```
that is PASS only if both fixture jpg files exist (SKIP otherwise, detail
says what to run). Hardware rows (SKIP unless hardware): ask the human 'Look
at the camera and stay still for 3 s', collect 60 frames, PASS if at least 90
percent of frames have exactly one face, all frames share one track_id, and
the mean mouth_open is below 0.15; then ask 'Open your mouth wide and close
it 3 times in 5 s', PASS if max minus min mouth_open is above 0.15.
6. Run: python tools/gates/run_gate.py 3

DO NOT implement pose or gaze yet. Report back in the AGENTS.md format.
```

### 3-B. You capture the fixtures, then run the hardware gate

```
python tools/capture_sample.py face_frontal        (neutral mouth, looking at
the camera)
python tools/capture_sample.py face_mouth_open     (mouth wide open, looking
at the camera)
python tools/demo_live.py --stage faces
python tools/gates/run_gate.py 3 --hardware
```

### 3-C. VERIFY checks (U5 with N=3)

```
1. Write a throwaway script that loads tests/fixtures/face_frontal.jpg, runs
FaceTracker, and saves out/verify_landmarks.jpg with small filled circles at
landmarks 13 and 14 (red), 33 and 263 (green), and the bbox in white. Print
the file path. Ask the human to open it and confirm: red dots on the inner
upper and lower lip, green dots on the outer eye corners. Do not continue
until the human answers.
2. Run the id tracker fuzz test with 20 different seeds and report failures.
3. Grep face_tracker.py for COLOR_BGR2RGB. Confirm it is present and applied
before FaceMesh.process.
4. Print mouth_open for both fixtures and quality for the frontal one.
```

## Phase 4: head pose, gaze, addressee score

Goal: for each face, an angle estimate and a 0 to 1 score for 'is this person facing the robot'.

### 4-A. BUILD prompt

```
Start: read AGENTS.md, docs/SPEC.md (section 5: head_pose, gaze, addressee)
and src/trustsync/synth.py. Phase 4 task. Create exactly:

1. src/trustsync/vision/head_pose.py: constant POSE_MODEL (dict landmark
index -> (x,y,z), values from SPEC) and estimate_pose exactly as SPEC.
Convert normalized pts to pixels with (x*w, y*h). Return (nan, nan, nan) if
the distance between landmarks 33 and 263 is below 5 pixels, if any of the 6
landmarks is NaN, or if cv2.solvePnP fails.
2. src/trustsync/vision/gaze.py: gaze_off exactly as SPEC.
3. src/trustsync/addressee/classifier.py: directed_score exactly as SPEC.
Clip the result to [0,1].
4. tools/calibrate_pose.py: asks 'Look straight at the robot (the camera) for
5 s', collects yaw and pitch with FaceTracker for 5 s, uses the median of
frames with exactly one face, copies config.yaml to config.yaml.bak, then
writes yaw_offset_deg and pitch_offset_deg (the two medians) into config.yaml
with save_config, and prints the new values. It refuses to write if fewer
than 30 valid frames were collected.
5. tools/demo_live.py: add --stage pose. Draw for each face: yaw, pitch, roll
as integers with a degree sign, gaze_off to 2 decimals, the directed score as
a horizontal bar, and an arrow starting at landmark 1 whose horizontal length
is 3 px per degree of (yaw - yaw_offset) and vertical length 3 px per degree
of (pitch - pitch_offset), clipped to 120 px. Keep the arrow sign logic in
the demo only, never in estimate_pose.
6. Tests (seeded):
   tests/test_head_pose.py: _POSE_MODEL in synth.py equals POSE_MODEL in
head_pose.py exactly. For every combination yaw in (-40,-20,0,20,40), pitch
in (-20,0,20), roll in (-15,0,15): estimate_pose(make_pts(...)) is within 3
degrees of the input on each angle. With 0.5 px Gaussian noise added to the
pixel positions each angle stays within 5 degrees. Same result within 3
degrees for scale 0.5 and for center (0.3,0.6). All-zero pts gives three NaN.
pts containing NaN gives three NaN.
   tests/test_gaze.py: iris_ratio 0.5 gives 0.0; 0.3 gives 0.4; 0.9 gives
0.8; 0.0 gives 1.0 (all within 1e-6); all-zero pts gives 0.0.
   tests/test_addressee.py with spec_cfg.addressee (25 and 20):
(yaw,pitch,gaze) = (0,0,0) gives 1.0; (25,0,0) gives 1.0; (37.5,0,0) gives
0.5; (50,0,0) gives 0.0; (0,30,0) gives 0.5; (0,0,0.6) gives 0.7; gaze 1.2
gives 0.7; use_gaze False ignores gaze; any NaN gives 0.0; yaw_offset_deg 10
makes (10,0,0) give 1.0; the score never increases as |yaw| grows (check 0 to
90 in steps of 1); 10000 random inputs including inf stay inside [0,1].
7. tools/gates/gate_4.py: rows for the three test files. Hardware rows (SKIP
unless hardware): ask 'Look at the camera 3 s' then 'Turn your head about 40
degrees away for 3 s'; PASS if the median directed score is above 0.8 for the
first and below 0.5 for the second (use the offsets in config.yaml).
```

```
8. Run: python tools/gates/run_gate.py 4

DO NOT change earlier phases except for a bug proven by a failing test.
Report back in the AGENTS.md format.
```

**4-B. You calibrate, then run the hardware gate**

```
python tools/calibrate_pose.py
python tools/demo_live.py --stage pose
python tools/gates/run_gate.py 4 --hardware
```

**4-C. VERIFY checks** (U5 with N=4)

```
1. Print the 6-entry landmark to model-point table in head_pose.py next to
the one in SPEC section 5 and confirm they are identical.
2. Run the 45-combination angle grid and print the maximum absolute error per
angle.
3. Throw 10000 random float arrays (including NaN and inf) at directed_score
and gaze_off and confirm no exception and results in [0,1].
4. Ask the human to run python tools/demo_live.py --stage pose, turn the head
to their own left, and say whether the printed yaw goes positive or negative.
Record the answer as a note in PROGRESS.md. (This is informational; only the
absolute value is used.)
```

# Phase 5: lip-audio sync scorer (the core idea)

Goal: one pure function that says how well a mouth signal and an audio envelope move together, plus the attribution helper. Because it is pure, it is tested exhaustively on synthetic data before any real face is involved. Use Planning mode for this phase.

Two honest expectations to set before you start. First, correlating a smooth mouth signal with a smooth envelope over 1.5 to 2 seconds has a real chance-level floor: a mismatched but moving mouth can reach a correlation of 0.3 to 0.5. The main defence against replay is therefore the static-mouth check ( `min_lip_std` ), and the correlation threshold is calibrated on your recorded data in Phase 8, not guessed here. Second, the gate for this phase prints the measured positive and null distributions so you can see the separation instead of trusting a threshold.

**5-A. BUILD prompt**

```
Start: read AGENTS.md and docs/SPEC.md (section 5: sync/scorer.py) and
src/trustsync/synth.py. Phase 5 task. Create exactly:

1. src/trustsync/sync/scorer.py with sync_score and attribute implemented
EXACTLY as the numbered steps in SPEC section 5. Implementation notes: use
float64; implement Pearson by hand on each overlap (mean-subtract, divide by
the product of norms, return 0.0 when either norm is below 1e-8); loop over
lags and the two features in plain Python (about 26 small correlations is
fine); if any input contains NaN return SyncResult(0.0, 0.0, 0.0, False, 0.0,
0.0); raise ValueError if lip and env lengths differ. Lag sign: a positive
lag means the audio is LATER than the lip motion: for j >= 0 pair f[:N-j]
with e[j:], for j < 0 pair f[-j:] with e[:N+j]. attribute breaks ties by
choosing the lowest track id.
2. tests/test_sync_scorer.py. Write a helper make_pair(rng, lag_s, noise,
dur_s=2.0, fps=30) that builds env with synth.make_env(dur_s+1, 100, [(0.3,
dur_s+0.7)], rng), resamples it to the fps grid starting at t=0.5 with
np.interp, builds lip with synth.make_lip(env, 100, 30, lag_s, 0.05, noise,
rng) and cuts both to the same N = round(dur_s*fps) samples starting at the
same index. Use cfg = spec_cfg.sync. Tests, each seeded per case:
   a. Positive: lag 0, noise 0.003, 200 seeds: score above 0.6 in at least 95
percent of seeds.
   b. Lag recovery: for lag_s in (-0.1, -0.0667, 0, 0.0667, 0.1), 100 seeds
each, noise 0.003: in at least 90 percent of seeds |estimated lag_s - lag_s|
<= 1/30. The sign of the estimate equals the sign of the input lag.
   c. Sign unit test, no randomness: lip = 0.02 + 0.05*exp(-((i-20)**2)/8),
env = 0.002 + 0.1*exp(-((i-23)**2)/8) for i in range(45); the result has
lag_s equal to +0.1 within 1e-9 (3 frames) and score above 0.7 (the log on
the envelope broadens its bump, so do not demand more).
   d. Static mouth: synth.make_static_lip noise 0.001, 200 seeds: score is
exactly 0.0, valid is True.
   e. Mismatched but moving: lip from one make_pair, env from a DIFFERENT
independent make_pair (different rng), 500 seeds: collect the scores as null;
collect 500 matched scores as pos (lag 0, noise 0.003). Assert AUC(pos vs
null) >= 0.92 and median(pos) - percentile95(null) >= 0.15. Compute AUC with
the rank formula inside the test (no sklearn needed).
   f. Noise monotonicity: median score over 100 seeds decreases (non-
strictly) for noise 0.003, 0.01, 0.02, 0.04; print the four medians.
   g. Degenerate: N below round(min_window_s*fps) gives valid False and score
0; constant env gives score 0 and no NaN anywhere in the result; all-zero env
gives score 0; NaN in lip gives valid False; unequal lengths raise
ValueError.
   h. Invariance: multiplying env by 7 leaves the score unchanged within 1e-
9; multiplying lip by 3 and adding 5 leaves the score unchanged within 1e-9
```

```
(use lip amplitudes large enough to pass the min_lip_std check).
   i. attribute: {1:0.7, 2:0.2} gives (1, 0.5); {3:0.6} gives (3, 0.6); {}
gives (None, 0.0); {4:0.5, 2:0.5} gives (2, 0.0).
   j. Multi-face: face A = matched lag 0.033 s (noise 0.003), face B = lip
from an independent pattern; 200 seeds: attribute picks A in at least 90
percent. With face B static instead: at least 99 percent.
   k. Speed: 1000 calls of sync_score with N=60 finish in under 1.5 s.
3. tools/gates/gate_5.py: a row for tests/test_sync_scorer.py (subprocess
pytest), and a row 'distributions' that recomputes pos and null (500 seeds
each, same recipe as test e) and prints in DETAIL: pos p5 / median, null
median / p95 / p99, AUC. PASS if the same assertions as test e hold.
4. Run: python tools/gates/run_gate.py 5

If a statistical test fails: report the printed numbers and STOP. DO NOT
change tolerances, thresholds, the algorithm in SPEC, or the synthetic
generators to force a pass. Report back in the AGENTS.md format.
```

**5-B. VERIFY checks** (U5 with N=5; this is the most important verification in the project)

```
1. In a throwaway script (not committed) write a NAIVE reference
implementation of sync_score straight from the numbered steps of SPEC section
5, using explicit Python loops and numpy.corrcoef on each overlap. Generate
50 random (lip, env) pairs with different lags and noise and show that the
maximum absolute difference between the reference score and src sync_score is
below 1e-6, and that lag_s agrees exactly.
2. Run the sign unit test by hand: delay the audio by 5 frames and report
lag_s; delay the lip by 5 frames (negative lag) and report lag_s. State in
one sentence which direction positive means and confirm it matches the
docstring in scorer.py.
3. Print the gate_5 'distributions' DETAIL line.
4. Try three adversarial inputs and report the result for each: (a) lip and
env both pure sine waves of the same 4 Hz frequency (periodic ambiguity), (b)
lip equal to env exactly, (c) env spikes of 1000x for one sample. State
whether the output is sensible.
5. Confirm min_lip_std is read from cfg and not hard-coded.
```

## Phase 6: pipeline, trust gate, overlay, offline runner

Goal: everything wired together behind one `process(frame, t)` call, tested end to end with scripted fake faces and synthetic audio, so no recording is needed yet. Use Planning mode. If this prompt is too big for the model, split it after step 4: send steps 1 to 4 as 6-A1

and steps 5 to 10 as 6-A2 (repeat the first line of the prompt in both), and run the tests you have between them.

### 6-A. BUILD prompt

```
Start: read AGENTS.md and docs/SPEC.md in full (sections 4, 5, 6 especially) and look at tests/conftest.py. Phase 6 task. Create exactly:

1. src/trustsync/fusion/gate.py: class Gate (decide method raising NotImplementedError) and RuleGate(cfg: Config) implementing SPEC section 6 in that exact order. All thresholds from cfg.gate and cfg.sync.min_lip_std. The reason string is one sentence containing the numbers that caused the decision, for example: 'best sync 0.12 below min 0.35 and lip motion 0.002: audio not explained by any visible mouth'.
2. src/trustsync/utterance.py: class UtteranceTracker(ucfg: UtteranceCfg, trim_s: float). update(t: float, speech: bool, obs: dict[int, dict[str, float]]) -> UtteranceEvent | None, where obs maps track_id to {'directed','quality','yaw','pitch'}. It starts an utterance when speech becomes True and records every frame's obs. It ends the utterance when speech becomes False OR when t - t_start >= ucfg.max_s; after a forced end it ignores speech until speech has been False once. On end it returns UtteranceEvent(t_start, t_end, kept, per_track) where t_end = time of the last frame with speech True minus trim_s, kept = (t_end - t_start) >= ucfg.min_s, per_track = {track_id: {name: list of floats}}. Otherwise it returns None. Property active.
3. src/trustsync/io/video_source.py: WebcamSource(cfg) and FileVideoSource(video_path, frame_times_csv) as in SPEC. WebcamSource sets width, height, fps from cfg, sets CAP_PROP_BUFFERSIZE to 1, and returns t = time.monotonic() right after read. FileVideoSource raises ValueError at construction if the csv row count differs from the video frame count; after the last frame read() returns (False, None, last_t).
4. src/trustsync/pipeline.py: TrustSyncPipeline exactly as SPEC section 5 (pipeline and utterance). Helper functions inside the module: _window_sync(buffer, t_now, audio, cfg) and _span_sync(buffer, t0, t1, audio, cfg). Both build the uniform grid t0 + arange(N)/fps with N = round((t1 - t0)*fps), interpolate lip with np.interp over the buffer, get env with audio.envelope_at(grid), and call sync_score. For _span_sync a track whose buffer covers less than 80 percent of the grid points returns SyncResult(0,0,0,False,0,0). Per-track buffers are collections.deque pruned to lip.buffer_s seconds; delete a track's buffer when its newest sample is older than lip.buffer_s. FrameResult.fps is the exponential moving average (alpha 0.1) of 1/(time.perf_counter() difference between consecutive process calls). While an utterance is active: decision LISTENING, reason 'listening',
```

```
and speaker_id, liveness, addressee, attribution_conf come from the running
per-frame scores (attribute() on running sync). Frames that are not part of
an utterance: IDLE with empty reason. The frame on which the utterance ends
carries the final GateDecision and utterance_final=True. A discarded short
utterance gives IDLE.
5. src/trustsync/overlay/hud.py: draw_hud(frame_bgr, result, env_hist) ->
np.ndarray, returning a copy. Scale all sizes by frame_height/480. Colours
(BGR): accept (191,212,45), reject (113,113,248), listening (36,191,251),
idle (139,116,100), text (243,237,230). Draw: (a) per face a corner-bracket
box (only the 4 corners, 2 px lines, arm length 15 percent of the box side)
in accept colour for result.speaker_id, in reject colour if the decision is
REJECT_* and the face is the best-sync face, otherwise idle colour; above it
a label 'id N  sync 0.00' on a dark translucent chip; below it a horizontal
bar of width = box width and fill = sync clipped to [0,1]; and a short arrow
from the box centre with length 3 px per degree of yaw (horizontal) and pitch
(vertical), clipped to 100 px. (b) A translucent dark panel top-left (alpha
0.55) with the decision text in its colour (font scale 0.9), the reason
truncated to 70 characters, 'speaker: N' or '-', and 'FPS 00'. (c) A bottom
strip 40 px high with a polyline of the last 200 values of env_hist, drawn in
the listening colour while speech_active else idle colour. Use
cv2.FONT_HERSHEY_SIMPLEX and cv2.LINE_AA. Clip boxes that fall outside the
frame; never crash on zero faces, an empty env_hist, or NaN values.
6. tools/run_offline.py: arguments --session PATH, --out DIR (default
out/<session id>), --video-out (flag). Loads meta.json,
FileVideoSource(video.mp4, frame_times.csv), FileAudioSource(audio.wav,
t0=meta['audio_t0'], cfg.audio), builds the pipeline with RuleGate, runs all
frames, writes DIR/frames.csv (columns
frame_idx,t,decision,speech_active,env,speaker_id,liveness,addressee,n_faces,
fps), DIR/utterances.json (list of frame_result_to_dict for frames with
utterance_final), and with --video-out also DIR/annotated.mp4 using draw_hud.
Prints one summary line: utterances=N decisions=[...] expected=<meta
expected>.
7. tools/demo_live.py: add --stage full. Opens WebcamSource and
LiveAudioSource, runs the pipeline, shows draw_hud in a window, prints a line
to the console on every utterance_final frame (time, decision, speaker,
liveness, addressee, reason). Keys: q quits, r rebuilds the pipeline (re-
calibrates the VAD).
8. Tests (all with spec_cfg and seeded rng; world helper below):
   World helper in tests/helpers_world.py: build_world(rng, bursts, faces)
where faces is a list of dicts {track_id, bbox, yaw, quality, mode} with mode
one of 'matched' (lip from synth.make_lip with lag 0.033, gain 0.05, noise
0.003), 'static' (synth.make_static_lip noise 0.001), 'independent' (lip from
a separate make_env pattern). Duration 8 s, fps 30, frame times 100.0 + k/30,
```

```
env at 100 Hz from t0 = 100.0 via NumpyAudioSource. For each frame k it
builds one FaceObs per face with pts = synth.make_pts(yaw=yaw,
mouth_gap_ratio=lip[k]) and mouth_open = lip[k]. It returns (pipeline,
frames, times) where the pipeline uses FakeFaceTracker. A run_world(...)
function feeds every frame (dummy black 480x640 BGR frame) and returns the
list of FrameResult.
   tests/test_gate.py: one test per rule 1 to 7 of SPEC section 6, built
directly from UtteranceSummary objects, plus a precedence test (a summary
that violates rules 2 and 3 gives REJECT_LOW_QUALITY), plus a test that
changing cfg.gate.min_sync changes the outcome.
   tests/test_utterance.py: a 1.0 s speech run is kept and its t_end is the
last speech frame minus trim_s; a 0.2 s run is not kept; a run longer than
max_s is force-ended at max_s and a second event does not fire until speech
has gone False once; two separate runs give two events.
   tests/test_video_source.py: write a 30-frame 64x48 mp4 with
cv2.VideoWriter('mp4v') and a matching csv in tmp_path; FileVideoSource
returns 30 frames with the csv times and then (False, None, last_t); a csv
with 29 rows raises ValueError. A hardware-marked test reads 10 frames from
WebcamSource with strictly increasing t.
   tests/test_hud.py: output shape equals input, input unchanged, output
differs from input, every decision in DECISIONS renders, zero faces and empty
env_hist do not crash, a face with bbox partly outside [0,1] does not crash,
NaN yaw does not crash.
   tests/test_pipeline_synthetic.py using the world helper, bursts [(3.0,
4.5)] unless stated:
     live_single (1 face matched, yaw 0, quality 1): exactly one frame has
utterance_final, its decision is ACCEPT with speaker_id 0, and the frames
between speech start and that frame are LISTENING.
     replay_spoof (1 face static): the final decision is REJECT_SPOOF.
     multi_static_other (face 0 left static, face 1 right matched): ACCEPT
with speaker_id 1.
     multi_independent_other, 20 seeds: never ACCEPT with speaker_id 0; at
least 60 percent of seeds end ACCEPT with speaker_id 1; the rest are
REJECT_AMBIGUOUS.
     not_addressed (1 face matched, yaw 50): REJECT_NOT_ADDRESSED.
     low_quality (quality 0.2): REJECT_LOW_QUALITY.
     no_face (tracker script all empty): REJECT_NO_FACE.
     silence (bursts []): no utterance_final frame, every decision IDLE.
     short_click (burst of 0.2 s): no utterance_final frame.
     calibration: no LISTENING frame earlier than 100.0 + vad.calib_s.
     robustness: a face whose pts are all NaN for 10 frames does not raise
and has directed 0.
     memory: 3000 frames with tracemalloc show growth below 50 MB.
```

```
   tests/test_e2e_pseudo_session.py (skip with pytest.skip if
tests/fixtures/face_frontal.jpg is missing): in tmp_path build a pseudo
session from the frontal photo: video.mp4 = the photo repeated for 240 frames
at 30 fps (640x480), frame_times.csv with t = 100 + k/30, audio.wav = 16000
Hz float32 made of white noise multiplied by synth.make_env(8,100,
[(3.0,4.5)],rng) upsampled to 16000 Hz, meta.json with audio_t0 100.0 and
expected REJECT_SPOOF. Run tools/run_offline.py through subprocess; assert
utterances.json has exactly 1 entry whose decision is REJECT_SPOOF (a static
real face with audio is the replay attack).
9. tools/gates/gate_6.py: one row per test file; hardware rows (SKIP unless
hardware): run demo-style pipeline for 20 s on the live webcam and microphone
while the human says one sentence facing the camera; PASS if no exception,
mean FPS is at least 15, and at least one utterance_final occurred (the
decision itself is NOT judged here; thresholds are calibrated in Phase 8).
Detail shows FPS and the decisions seen.
10. Run: python tools/gates/run_gate.py 6

DO NOT calibrate or change any threshold in config.yaml. DO NOT add features
beyond this list. Report back in the AGENTS.md format.
```

### 6-B. You run it live

```
python tools/demo_live.py --stage full
python tools/gates/run_gate.py 6 --hardware
```

Speak a sentence facing the camera, then hold your phone near your closed mouth and play a recorded voice. Do not worry if an accept fails yet; thresholds are uncalibrated. What you are checking is that the overlay works and an utterance decision is printed each time.

**6-C. VERIFY checks** (U5 with N=6)

```
1. Run each synthetic scenario from tests/helpers_world.py in a throwaway
script and print, per scenario: time of first LISTENING frame, time of the
utterance_final frame, final decision, speaker_id, liveness,
attribution_conf, addressee.
2. Prove thresholds come from config: load spec_cfg, replace gate.min_sync
with 0.99 using dataclasses.replace, rerun live_single, and show that the
decision becomes REJECT_SPOOF.
3. Confirm there is exactly one utterance_final frame per utterance in 5
different seeds of live_single.
4. Confirm the pipeline never keeps more than lip.buffer_s seconds per track
(print the length of a buffer after 3000 frames).
5. Run tools/run_offline.py on the pseudo session from the e2e test (keep it
in out/verify_pseudo) with --video-out, print the first 5 rows of frames.csv,
and report the size of annotated.mp4.
6. Ask the human to open out/verify_pseudo/annotated.mp4 and say whether
boxes, the decision panel and the envelope strip are visible and readable.
```

## Phase 7: recorder, validator, recording protocol

Goal: tools that capture clean, labelled, time-aligned sessions, and a protocol your three-person team follows so the evaluation data is balanced. The code part is quick; the recording part is the real work (plan 90 minutes).

### 7-A. BUILD prompt

```
Start: read AGENTS.md and docs/SPEC.md (sections 5 io and 7). Phase 7 task.
Create exactly:

1. src/trustsync/io/recorder.py: class SessionRecorder(cfg: Config, out_dir:
Path, meta: dict, video=None, audio=None). If video is None use
WebcamSource(cfg); if audio is None use LiveAudioSource(cfg.audio,
record_to=out_dir/'audio.wav'). run(duration_s: float, preview: bool = True)
-> Path: start the audio source first, then loop reading frames; write each
frame to out_dir/video.mp4 with cv2.VideoWriter('mp4v', cfg.video.fps,
(width, height)) (size taken from the first frame) and remember (frame_idx,
t); with preview show a window with a red REC dot, elapsed and remaining
seconds, and on the first line the command text in meta['prompt_text'] when
present; stop after duration_s or on key q or when read() returns ok False;
close the audio; write out_dir/frame_times.csv (header frame_idx,t) and
out_dir/meta.json = the given meta plus audio_t0 = audio.t0 and video_fps =
(n-1)/(t_last - t_first). Return out_dir. Create out_dir if needed; refuse
(raise FileExistsError) if video.mp4 already exists.
2. tools/record_session.py. Arguments: --scenario (choices from SPEC section
7), --subjects (one or more names), --speaker-side (left|right|center|none,
default center), --addressed (true|false, default true), --lighting
(normal|dim, default normal), --occlusion (true|false, default false), --
spoof-device (none|phone, default none), --duration (seconds, default 7), --
count (how many clips to record in a row, default 1), --notes. It fills
meta.json keys exactly as SPEC section 7 (expected derived from the scenario:
live_single ACCEPT, replay_spoof REJECT_SPOOF, multi_one_speaks ACCEPT,
not_addressed REJECT_NOT_ADDRESSED, occlusion_lowlight REJECT_ANY, silence
NONE). Session id = <scenario>_<first subject lower-case>_<3-digit counter>
where the counter is the next free number in data/sessions. Prompt texts
cycle through this list (shown on screen as 'Say: ...'; not shown for silence
or replay_spoof): robot, bring me water | stop | turn left | what time is it
| open the door | call the nurse | turn on the light | go to the kitchen |
play some music | follow me. Before each clip show a 3 second countdown in
the preview window. After each clip run the validator and print its table; if
the validator fails, print 'RE-RECORD THIS CLIP' and delete nothing.
3. tools/validate_session.py: function validate(path: Path, check_faces: bool
= True) -> list[tuple[str,str,str]] (check, PASS or FAIL, detail) and a CLI
that prints the table and exits 1 on any FAIL (usage: python
tools/validate_session.py data/sessions/<id> or --all). Checks: the 4 files
exist; meta.json has all SPEC section 7 keys with legal values and expected
consistent with scenario; frame_times strictly increasing and its row count
equals cv2 CAP_PROP_FRAME_COUNT; measured fps between 15 and 35; audio is
16000 Hz mono; |audio duration - video duration| < 0.3 s; |audio_t0 - first
frame t| < 0.5 s; audio peak below 0.99 (no clipping); for every scenario
```

```
except silence: VAD (segment_speech on the file envelope) finds at least one
segment of 0.4 s or more starting after 1.0 s; for silence: no segment; if
check_faces: run FaceTracker on every 5th frame and require a face in at
least 70 percent of them (skip this check for occlusion_lowlight); for
multi_one_speaks require at least 2 faces in at least 60 percent of the
sampled frames.
4. Move the pseudo-session builder out of tests/test_e2e_pseudo_session.py
into tests/helpers_world.py as make_pseudo_session(tmp_path, rng,
scenario='replay_spoof') and keep that test working.
5. Tests: tests/test_validate_session.py: the pseudo session passes
validate(check_faces=False) (its video is a photo, so also run once with
check_faces=True and expect PASS when the fixture exists). Each of these
mutations of a copy makes validate FAIL with a detail naming the problem:
delete a meta key; shuffle two rows of frame_times.csv; rewrite audio.wav at
44100 Hz; add 5.0 to audio_t0; replace the audio with zeros for a live_single
scenario; drop the last row of frame_times.csv. tests/test_recorder.py: with
a FakeVideo (60 frames of black 480x640, t = 100 + k/30) and a FakeAudio
(start() writes a 2 s 16 kHz noise WAV to its path, attribute t0 = 100.0,
close() does nothing), SessionRecorder.run(2.0, preview=False) writes all
four files, frame_times has 60 rows and meta.json contains audio_t0 100.0 and
video_fps within 1 of 30; a second run into the same folder raises
FileExistsError.
6. tools/gates/gate_7.py: rows for the two test files; hardware row (SKIP
unless hardware): ask the human to 'say: robot, stop' and record a 10 s
session with scenario live_single and subject gatetest into a temporary
folder through SessionRecorder, then validate it; PASS only if every
validator row passes; the detail names the failing rows.
7. Run: python tools/gates/run_gate.py 7

DO NOT record real data in this prompt. Report back in the AGENTS.md format.
```

### 7-B. VERIFY checks (U5 with N=7)

```
1. Run python tools/validate_session.py on each mutated copy from the tests
(re-create them in a throwaway script) and paste the 6 failing detail
strings. Each must name a specific cause.
2. Confirm record_session.py refuses to overwrite and that the session id
counter skips existing folders (create two empty folders named
live_single_x_001 and live_single_x_002 inside a temporary data directory and
show the next id is 003).
3. Run python tools/gates/run_gate.py 7 --hardware with the human speaking
and paste the table.
```

## Recording protocol (you and your two teammates)

Record at a fixed spot: webcam at eye level, face about 60 cm away, a quiet room, the phone speaker at normal volume. Each clip is 7 seconds: stay silent for the first 1.5 s (the VAD calibrates), then say the prompted command once. Aim for the minimum column first; add the full column if time allows. Always record the live clips before the replay clips, because the replay clips play back the person's own live audio.

| Scenario | How to record it | Minimum clips | Full set |
|---|---|---|---|
| `live_single` | One person faces the camera and says the command. | 5 per person (15) | 8 per person (24) |
| `replay_spoof` | The same person sits in frame with mouth closed and still; a phone about 10 cm below the chin plays that person's own `live_single` audio (copy `audio.wav` to the phone). | 5 per person (15) | 8 per person (24) |
| `multi_one_speaks` | Two people side by side, only one speaks; the other stays silent but alive (blinks, shifts, small smile). Use `--speaker-side left` or `right`. Swap who speaks and sides. | 2 per pair (6) | 4 per pair (12) |
| `not_addressed` | One person speaks while turned 45 to 60 degrees away, toward the other person or the wall. Use `--addressed false`. | 5 per person (15) | 8 per person (24) |
| `occlusion_lowlight` | Mouth fully covered by hand or mask, or a nearly dark room lit only by a monitor. Use `--occlusion true` and `--lighting dim` as fits. These must all be rejected. | 3 per person (9) | 6 per person (18) |
| `silence` | Faces in frame, nobody speaks; keyboard taps and room noise only. | 4 | 6 |

Minimum total: 64 clips. Full total: 108 clips. Example commands:

```
python tools/record_session.py --scenario live_single --subjects pratik --
count 8
python tools/record_session.py --scenario multi_one_speaks --subjects pratik
suchet --speaker-side left --count 2
python tools/record_session.py --scenario not_addressed --subjects suchet --
addressed false --count 8
```

Set the subject names once and keep the spelling identical across clips, because Phase 8 holds out one subject at a time to test generalization. Run `python tools/validate_session.py --all` at the end and re-record any clip that fails.

## Phase 8: evaluation, calibration, learned gate

Goal: turn your recorded clips into the numbers in your Evaluation Plan (accuracy, liveness precision and recall, false-accept rate, attribution accuracy, addressee accuracy), calibrate the thresholds honestly with leave-one-subject-out testing, and train the scikit-learn trust classifier. This phase has three BUILD prompts; run them in order with a gate between each. All three can be built and tested on synthetic records before you finish recording; the final real run needs the dataset from Phase 7.

How the design avoids re-running vision: each clip is processed once and cached as a `ClipRecord` (the per-track utterance features, which do not depend on gate thresholds). Threshold sweeps then take seconds. Calibration objective: keep the false-accept rate on should-reject clips at or below 2 percent, then maximize accept/reject accuracy.

**8-A1. BUILD prompt: records, cache, metrics**

```
Start: read AGENTS.md and docs/SPEC.md (sections 4, 6, 7). Phase 8 part 1.
Create exactly:

1. src/trustsync/eval/metrics.py containing:
   dataclass UttRecord(t_start: float, t_end: float, summary:
UtteranceSummary, face_cx: dict[int,float])  (face_cx = bbox centre x per
track at the utterance_final frame)
   dataclass ClipRecord(id: str, scenario: str, subject: str, subjects:
list[str], expected: str, speaker_side: str, addressed: bool, utts:
list[UttRecord], lag_samples: list[float])  (subject = subjects[0];
lag_samples = best_lag_s of the highest-sync face on frames that are
LISTENING and have that face sync above 0.5)
   dataclass ClipOutcome(decision: str, speaker_id: int|None, speaker_side:
str|None, summary: UtteranceSummary|None, gd: GateDecision|None)
   side_of(cx: float) -> str: left if cx < 0.4, right if cx > 0.6, else
center.
   records_to_json(records) -> str and records_from_json(text) ->
list[ClipRecord] (lossless, NaN stored as null).
   decide_clip(rec: ClipRecord, gate: Gate) -> ClipOutcome: take the
utterance with the longest t_end - t_start, call gate.decide on its summary;
no utterance gives ClipOutcome('NONE', None, None, None, None). speaker_side
= side_of(face_cx[speaker_id]) when speaker_id is not None.
   auc(pos: list[float], neg: list[float]) -> float (rank formula, ties count
half).
   compute_metrics(records, outcomes: list[ClipOutcome], cfg: Config) -> dict
with these keys: n_clips; decision_accuracy; far_spoof; far_all_reject; frr;
false_trigger_rate; missed_utterance_rate; liveness {precision, recall, f1};
attribution_accuracy; addressee_accuracy; reason_accuracy; per_scenario
{scenario: {n, accuracy, accept_rate}}; confusion {expected: {decision:
count}}.
   Definitions (write them as docstrings): a clip is 'should accept' if
expected == ACCEPT, 'should reject' if expected in (REJECT_SPOOF,
REJECT_NOT_ADDRESSED, REJECT_ANY), and silence (expected NONE) must produce
no utterance. decision_accuracy = share of clips where accept/reject agrees
(for NONE: outcome decision is NONE); far_spoof = share of replay_spoof clips
whose decision is ACCEPT; far_all_reject = share of should-reject clips
accepted; frr = share of should-accept clips not accepted; false_trigger_rate
= share of silence clips with at least one utterance; missed_utterance_rate =
share of non-silence clips with no utterance; liveness uses positive = live,
truth live for scenarios live_single, multi_one_speaks, not_addressed and not
live for replay_spoof (other scenarios excluded), predicted live = best-sync
track has sync >= cfg.gate.min_sync and lip_std >= cfg.sync.min_lip_std (no
utterance = not live); attribution_accuracy = over multi_one_speaks clips
```

```
with an utterance, the share where the outcome speaker_side equals the clip
speaker_side (independent of accept); addressee_accuracy = over live_single
and not_addressed clips with an utterance, share where (best-sync track
directed >= cfg.gate.min_directed) equals clip.addressed; reason_accuracy =
over replay_spoof and not_addressed clips, share where the decision string
equals expected. Return 0.0, never NaN or a ZeroDivisionError, when a
denominator is 0.
2. src/trustsync/eval/run_eval.py (only the function part now):
process_session(session_dir: Path, cfg: Config) -> ClipRecord, using
FileVideoSource, FileAudioSource, TrustSyncPipeline and a class
RecordingGate(Gate) that wraps RuleGate and appends every summary it is asked
to decide to self.summaries; and load_or_process(session_dir, cfg, cache_dir)
that caches the record as cache_dir/<id>.json and reuses it only if the file
is newer than every file in the session and a stored hash of cfg.vad,
cfg.utterance, cfg.tracker, cfg.lip, cfg.sync, cfg.quality, cfg.addressee,
cfg.video matches. Gate thresholds are deliberately NOT part of the hash.
3. Tests (hand-computable, no hardware): tests/test_eval_metrics.py with a
helper fake_clip(scenario, expected, best_sync, lip_std, directed,
quality=1.0, n_tracks=1, speaker_side='center', addressed=True, subject='a',
speaker_cx=0.5, duration=1.5). Build a set of 12 clips with known outcomes
under spec_cfg and RuleGate and assert exact values: decision_accuracy,
far_spoof (with one of two replay clips accepted: 0.5), frr, liveness
precision and recall, attribution_accuracy for a 3-clip multi set with 2
right, addressee_accuracy, reason_accuracy, false_trigger_rate,
missed_utterance_rate, per_scenario counts, confusion counts. Also: empty
list gives zeros; json round trip equals the original including NaN;
auc([1,2],[0,0]) == 1.0 and auc([1],[1]) == 0.5; side_of boundaries 0.4 and
0.6 give center. tests/test_eval_cache.py (skip if
tests/fixtures/face_frontal.jpg is missing): build a pseudo session with
tests/helpers_world.make_pseudo_session, call load_or_process twice and check
the second call does not call process_session (use monkeypatch to count),
then change cfg.sync.window_s with dataclasses.replace and check it re-
processes; changing cfg.gate.min_sync does NOT re-process.
4. tools/gates/gate_8.py: add rows for these two test files now (later
prompts add more rows to the same file).
5. Run: python tools/gates/run_gate.py 8

Report back in the AGENTS.md format.
```

## 8-A2. BUILD prompt: calibration and learned gate

```
Start: read AGENTS.md, docs/SPEC.md and src/trustsync/eval/metrics.py. Phase
8 part 2. Create exactly:

1. src/trustsync/eval/calibrate.py:
   sweep_rule_thresholds(records, cfg, max_far=0.02) -> tuple[Config,
list[dict]]: grid min_sync in 0.05..0.60 step 0.05 (12 values), min_margin in
(0.0, 0.05, 0.10, 0.15, 0.20), min_directed in (0.3, 0.4, 0.5, 0.6, 0.7,
0.8), sync.min_lip_std in (0.002, 0.003, 0.004, 0.006, 0.008, 0.012). For
each combination build cfg with dataclasses.replace and score decisions with
RuleGate over outcomes (call decide_clip once per record per combination).
Objective: among combinations with far_all_reject <= max_far choose the
highest decision_accuracy; if none satisfies the constraint choose the lowest
far_all_reject then highest accuracy. Then take every combination whose
accuracy is within 0.01 of the best and satisfies the same FAR rule and
return the one nearest to the per-parameter median of that set (plateau
centre). Return the new cfg and the full table (list of dicts). min_quality
is NOT swept.
   loso(records, fit_fn, predict_fn, cfg) -> dict: leave-one-subject-out by
ClipRecord.subject. fit_fn(train_records, cfg) returns a model object;
predict_fn(model, test_records, cfg) returns a list of ClipOutcome. Returns
{'folds': {subject: metrics}, 'pooled': compute_metrics on all held-out
outcomes}. A subject's records must never be used in its own fit (assert it).
   estimate_av_lag(records, cfg) -> dict(median, p10, p90, n,
saturated_fraction) from lag_samples, where saturated means abs(lag) >=
cfg.sync.max_lag_s - 1/cfg.video.fps. median is None and saturated_fraction
0.0 when there are no samples.
2. src/trustsync/fusion/learned.py: FEATURE_NAMES =
['sync_best','sync_second','margin','lip_std_best','directed_best','abs_yaw_b
est','abs_pitch_best','quality_max','n_tracks','duration_s'];
summary_features(summary) -> np.ndarray shape (10,) (best = highest sync
track; second = 0 if one track; NaN yaw or pitch become 90);
train_learned_gate(records, cfg, kind='logreg', max_far=0.02) -> dict with
keys model, threshold, feature_names, kind, using one sample per clip (its
longest utterance; clips with no utterance, no track, or expected NONE are
skipped), label 1 for expected ACCEPT else 0; kind 'logreg' = sklearn
Pipeline(StandardScaler, LogisticRegression(C=1.0, class_weight='balanced',
max_iter=1000)); kind 'gboost' = GradientBoostingClassifier(n_estimators=100,
max_depth=2, random_state=1234); threshold = the probability cut-off from a
scan of 0.05..0.95 step 0.01 that keeps training FAR <= max_far and maximizes
training accuracy (if none, the lowest-FAR cut-off); save_gate(bundle, path)
and load with joblib. class LearnedGate(Gate): __init__(cfg) loads
cfg.gate.model_path (FileNotFoundError if missing). decide(summary): same
steps 1 and 2 as RuleGate (no tracks gives REJECT_NO_FACE, max quality below
```

```
min_quality gives REJECT_LOW_QUALITY); otherwise p = predict_proba(features)
[1]; p >= threshold gives ACCEPT with speaker_id = best track; else a reject
whose decision is the first failing rule among RuleGate steps 3, 4, 6
evaluated with cfg thresholds, or REJECT_AMBIGUOUS if none fails, and the
reason sentence includes the probability. Add to src/trustsync/fusion/gate.py
(additive only): make_gate(cfg: Config) -> Gate returning RuleGate when
cfg.gate.mode == 'rule' and LearnedGate when 'learned' (import LearnedGate
inside the function).
3. Tests (synthetic ClipRecords via the fake_clip helper from
tests/test_eval_metrics.py, moved into tests/helpers_eval.py):
tests/test_calibrate.py: perfectly separable data (live: sync 0.8 directed
0.9; replay: sync 0.05 lip_std 0.001; not addressed: sync 0.8 directed 0.1)
gives a calibrated cfg with accuracy 1.0 and FAR 0.0 and thresholds strictly
between the class values; overlapping data (sync drawn from normal(0.55,0.15)
for live and normal(0.30,0.15) for replay, rng seeded, 60 clips each) gives
FAR <= 0.02 on the training data; loso on 3 subjects calls fit_fn 3 times,
never with the held-out subject (use a spy), and pooled n_clips equals the
total; estimate_av_lag recovers a known median within 0.01 and flags
saturation for lags at the edge; empty input handled. tests/test_learned.py:
feature vector length 10 and correct values for a hand-built summary; trained
logreg on separable data reaches training accuracy 0.95 or more; save then
load gives identical probabilities; LearnedGate returns REJECT_NO_FACE and
REJECT_LOW_QUALITY before touching the model; a clear spoof summary is
rejected and a clear live summary accepted; make_gate returns the right class
for both modes (write a model into tmp_path and point cfg.gate.model_path at
it).
4. tools/gates/gate_8.py: add rows for the two new test files.
5. Run: python tools/gates/run_gate.py 8

DO NOT write to config.yaml in this prompt. Report back in the AGENTS.md
format.
```

### 8-A3. BUILD prompt: ablation, plots, results.json, CLI

```
Start: read AGENTS.md, docs/SPEC.md, src/trustsync/eval/metrics.py,
calibrate.py and fusion/learned.py. Phase 8 part 3. Create exactly:

1. src/trustsync/eval/plots.py (matplotlib with the Agg backend; dark theme:
figure and axes facecolor #0A0E14, text and axes #E6EDF3, gridlines #FFFFFF
at 6 percent opacity, accept #2DD4BF, reject #F87171, listening #FBBF24, info
#A78BFA, idle #64748B; figsize (8,4.5), dpi 200, PNG, no title overlap, axis
labels with units): plot_roc(pos, neg, title, path);
plot_box_by_scenario(values_by_scenario: dict[str, list[float]], ylabel,
threshold: float|None, path); plot_confusion(confusion, path);
plot_ablation(rows, path) (grouped bars of accuracy and FAR per variant);
plot_lag_hist(lags, max_lag_s, path); plot_timeline(frames_csv: Path, title,
path) (rows of frames.csv: env, liveness, addressee over time with
speech_active shaded and the decision colour at the bottom). Each returns the
path and creates parent folders.
2. In src/trustsync/eval/run_eval.py add: ablation gate classes AcceptAll
(B0, the naive voice interface: ACCEPT if any utterance), SyncOnly (A1),
SyncMargin (A2), SyncMarginAddressee (A3), and use RuleGate as A4 and
LearnedGate as A5; build_ablation(records, cfg, learned_bundle) -> list of
dicts {name, accuracy, far_all_reject, frr, liveness_f1}. And main(argv):
arguments --sessions DIR (default data/sessions), --records-json FILE (skip
processing, load records), --out DIR (default reports), --cache DIR (default
out/cache), --calibrate, --learned, --write-config. Flow: load or process
records; metrics with the current cfg (key 'current'); if --calibrate: sweep
on all records (key 'calibrated' plus the thresholds) and loso with the sweep
(key 'loso_rule'); if --learned: train logreg and gboost, loso for both, keep
the one with the higher pooled accuracy subject to pooled far_all_reject <=
0.02 (else the one with lower FAR), save models/gate.joblib (key
'loso_learned', 'learned_kind'); ablation; estimate_av_lag; figures into
OUT/figures: roc_sync.png (live vs replay best sync), roc_directed.png
(addressed vs not), sync_by_scenario.png, confusion.png, ablation.png,
lag_hist.png, and timeline_<scenario>.png for the first clip of each scenario
when frames.csv exists in out/<id>/ (run tools/run_offline.py logic via a
function if needed); write OUT/results.json with keys generated_at (ISO),
dataset {n_clips, per_scenario_counts, subjects}, config {gate, min_lip_std,
addressee}, current, calibrated, loso_rule, loso_learned, learned_kind,
ablation, av_lag, fps (null for now), figures [{name, path, caption}], notes
(list of strings). With --write-config: copy config.yaml to config.yaml.bak,
write the calibrated gate thresholds and sync.min_lip_std with save_config,
set gate.mode to learned only if --learned and its pooled FAR is within the
limit, and print the diff.
3. Tests: tests/test_eval_run.py: using synthetic records written to tmp_path
as JSON, run main(['--records-json', path, '--out', tmp/'reports', '--
```

```
calibrate', '--learned']) with cwd tmp_path (models/ directory created
there): results.json exists and has every key listed above, ablation has 6
rows named B0, A1, A2, A3, A4, A5 in that order, B0 has far_all_reject 1.0 on
a set that contains should-reject clips, A4 has lower FAR than B0, all listed
figure files exist and are larger than 5 KB, and --write-config creates
config.yaml.bak (run it against a copy of config.yaml in tmp_path, never the
real one). tests/test_plots.py: each plot function creates a non-empty PNG
for tiny inputs and does not raise for empty inputs.
4. tools/gates/gate_8.py: add rows for the two new test files. Add a row
'dataset present' that is PASS when data/sessions has at least 40 sessions
that pass validate_session, SKIP otherwise (detail: record more clips). When
present add: row 'run_eval end to end' that runs python -m
trustsync.eval.run_eval --calibrate --learned into a temp output folder and
checks results.json keys; row 'AV lag not saturated' PASS if av_lag
saturated_fraction < 0.2 (detail: median lag and the advice 'set
audio.latency_s to the negative median if it is large'); and four GOAL rows
reading LOSO pooled numbers: far_all_reject <= 0.10, frr <= 0.30,
attribution_accuracy >= 0.85, addressee_accuracy >= 0.80. A failing GOAL row
means investigate with the human, never tune the test.
5. Run: python tools/gates/run_gate.py 8

DO NOT run --write-config on the real config.yaml in this prompt. Report back
in the AGENTS.md format.
```

**8-B. You run the real evaluation** (after recording; this changes `config.yaml`, so commit before)

```
python tools/validate_session.py --all
python -m trustsync.eval.run_eval --calibrate --learned --write-config
python tools/gates/run_gate.py 8 --hardware
```

If a GOAL row fails, the usual causes in order of likelihood are: clips recorded too dark or too far from the camera; speaking before the 1 s VAD calibration ended; `audio.latency_s` far off (see the lag histogram); a single subject whose mouth motion is subtle. Bring the printed numbers back to me before changing anything.

**8-C. VERIFY checks** (U5 with N=8)

```
1. Pick 5 clips from the cache covering 5 different scenarios. For each,
print the longest utterance summary (best track values), the gate decision
under the calibrated config, and the expected label. Recompute accept/reject
agreement by hand and compare with compute_metrics per_scenario accuracy.
2. Prove there is no leakage: read calibrate.loso and show the line that
removes the held-out subject before fit_fn is called. Run the spy test and
paste its output.
3. Open reports/results.json and confirm every key is present, no value is
NaN, and the numbers in 'current' and 'calibrated' differ only where
thresholds changed.
4. Confirm config.yaml.bak exists after --write-config and print the diff
between it and config.yaml; only gate.*, sync.min_lip_std and addressee
offsets may differ.
5. Confirm tests still read tests/fixtures/config_spec_defaults.yaml: run
pytest -q -m 'not hardware' and report the count of passed tests.
6. Ask the human to open reports/figures and say whether every axis is
labelled with units and every chart is readable on a dark background.
```

## Phase 9: FastAPI backend

Goal: the pipeline running in a background thread, with a raw MJPEG video stream, a WebSocket of results, and small control routes. The browser draws all overlays itself from the JSON, so the backend never burns graphics into the video. Use Planning mode.

### 9-A. BUILD prompt

```
Start: read AGENTS.md and docs/SPEC.md (sections 3, 4, 8). Phase 9 task.
Create exactly:

1. src/trustsync/server/runtime.py: class Runtime(cfg: Config, root: Path |
None = None, factory=None).
   - dataclass Sources(video, audio, tracker, realtime: bool). Default
factory build_sources(kind, session_id, cfg, root) -> Sources: kind 'webcam'
gives WebcamSource, LiveAudioSource(cfg.audio), tracker None (the pipeline
builds the real FaceTracker), realtime False; kind 'session' gives
FileVideoSource and FileAudioSource for root/data/sessions/<id> (t0 from
meta.json audio_t0), tracker None, realtime True. Raise FileNotFoundError for
a missing session and RuntimeError('cannot open camera') if the webcam does
not open.
   - Methods: start(kind, session_id=None) (stops any running source first,
builds Sources, creates TrustSyncPipeline(cfg, audio, make_gate(cfg),
tracker), starts a daemon thread); stop() (idempotent, joins the thread for
up to 2 s, closes sources); status() -> dict(running, source, error);
get_config_dict(); set_config(section, key, value) (only sections gate and
addressee, plus sync.min_lip_std; numeric values only, except
addressee.use_gaze which is a bool; gate thresholds must be within [0,1];
addressee values must be positive except the two offsets; raise ValueError
otherwise; builds a new Config with dataclasses.replace, runs validate, and
if a source is running rebuilds the pipeline between two frames under a lock,
which re-calibrates the VAD for about 1 s); latest_jpeg() -> bytes (a pre-
rendered 640x480 dark placeholder with the text No source when nothing runs);
subscribe() -> asyncio.Queue and unsubscribe(q).
   - Worker loop: read frame; result = pipeline.process(frame, t); encode the
RAW frame as JPEG with cv2.imencode at quality cfg.server.jpeg_quality and
store it; publish {type:'frame', data: frame_result_to_dict(result)} at most
cfg.server.ws_hz times per second (decided with time.monotonic), and ALWAYS
also publish {type:'utterance', data: ...} when utterance_final; when
realtime is True sleep so that frame k is processed at wall time start + (t_k
- t_0); on end of file or on an exception set running False, store the error
string, and publish {type:'status', data: status()}. Publishing uses
loop.call_soon_threadsafe(queue.put_nowait, msg) for the event loop captured
in subscribe(); each queue has maxsize 50 and when it is full the OLDEST
message is dropped, so a slow client can never block the worker.
2. src/trustsync/server/app.py: create_app(cfg: Config | None = None, root:
Path | None = None, runtime: Runtime | None = None) -> FastAPI with exactly
the routes of SPEC section 8. Details: GET /api/health returns {ok: true,
running, source}; POST /api/config body {section,key,value} returns the
updated dict, a ValueError becomes HTTP 422; GET /api/sessions lists
root/data/sessions/*/meta.json as [{id, scenario, expected, subjects}] sorted
```

```
by id; POST /api/source body {type, id?}: an unknown session gives 404, a
session id that does not match ^[A-Za-z0-9_-]+$ gives 422, a camera failure
gives 409 with the message, an unknown type gives 422; GET /api/results
returns root/reports/results.json or 404; GET /stream.mjpg streams
multipart/x-mixed-replace with boundary=frame, yielding runtime.latest_jpeg()
about every 1/ws_hz seconds (when the frame bytes are unchanged still yield
at least every 0.5 s); WS /ws sends the hello message, then every queued
message as JSON text, unsubscribes on disconnect, and must not raise on a
client that closes abruptly. CORS: allow origins http://localhost:5173 and
http://127.0.0.1:5173. If root/web/dist exists mount it at / as static files
with html=True, mounted LAST so API routes win. Shutdown calls
runtime.stop(). Add a module-level main() so that python -m
trustsync.server.app [--port N] runs uvicorn on cfg.server.host and the port.
3. Tests (no hardware, no real camera): tests/test_server.py using
fastapi.testclient.TestClient, a Runtime with a test factory that returns a
synthetic world from tests/helpers_world.py (live_single, 8 s, realtime
False, black frames, FakeFaceTracker, NumpyAudioSource), and a tmp_path root
containing data/sessions/s1/meta.json and s2/meta.json:
   health before start has running false; config GET has the sections gate,
addressee, sync; config POST gate.min_sync 0.5 is reflected in the next GET;
invalid cases give 422 (section video, value abc, gate.min_sync 5, unknown
key); sessions lists s1 and s2; source with id ../x gives 422 and an unknown
id gives 404; a failing factory (raises RuntimeError) gives 409 and health
shows running false; results gives 404, then 200 after writing
root/reports/results.json.
   WebSocket flow: connect to /ws, the first message has type hello with keys
config and decisions (8 names); POST /api/source with type session and id
synthetic; read messages until one of type utterance arrives (timeout 20 s):
its data decision is ACCEPT and speaker_id is 0; every frame message data has
exactly the keys of frame_result_to_dict(FrameResult); a status message with
running false arrives after the world ends.
   Throttle: during a non-realtime run of 240 frames the number of frame
messages received is at most elapsed*ws_hz + 5 (measure the elapsed time in
the test).
   Backpressure: subscribe a queue and never read it; run 500 fast frames;
the run finishes (running false) and queue.qsize() <= 50.
   MJPEG: with a running source, client.stream GET /stream.mjpg: the first
2000 bytes contain b'--frame' and the JPEG start bytes FF D8; with nothing
running the placeholder is served.
   Lifecycle: stop() twice does not raise; start while running replaces the
source; stop joins the thread within 2 s.
4. tools/gates/gate_9.py: a row for tests/test_server.py. Hardware rows (SKIP
unless hardware): start the server in a subprocess on a free port, POST
```

```
/api/source with the webcam type, read 20 WebSocket frame messages within 3
seconds, decode one JPEG from /stream.mjpg with cv2.imdecode and check its
shape is (height, width, 3) of the camera, POST /api/stop, terminate the
subprocess; PASS if all of that works.
5. Run: python tools/gates/run_gate.py 9

DO NOT draw any overlay on the streamed frames. DO NOT add routes that are
not in SPEC section 8. Report back in the AGENTS.md format.
```

**9-B. You run it by hand**

```
python -m trustsync.server.app
```

Open http://127.0.0.1:8000/docs in Chrome: the interactive API page should list every route. In it, run POST /api/source with the body type = webcam, then open http://127.0.0.1:8000/stream.mjpg and confirm you see your camera. Finally run `python tools/gates/run_gate.py 9 --hardware`.

**9-C. VERIFY checks** (U5 with N=9)

```
1. Send these requests with Python httpx against a running test app and paste
each status code: POST /api/source with id '../../etc/passwd'; id
'..%2F..%2Fsecret'; an id containing a null byte; a 10 KB id; type 'banana'.
None may return 200 or 500.
2. Write a throwaway websockets client that connects, prints the type and
decision of the first 5 messages after starting a synthetic session, and
disconnects abruptly (close the socket without a close frame). Confirm the
server log shows no traceback and the next client can still connect.
3. Run the synthetic world 20 times in a loop through runtime start and stop
and report the thread count before and after (threading.active_count()); it
must not grow.
4. Confirm CORS: an OPTIONS request with Origin http://localhost:5173 returns
the allow-origin header; with Origin http://evil.example it does not.
5. Confirm no route exists outside SPEC section 8 except the automatic /docs,
/openapi.json and /redoc.
```

## Phase 10: modern dashboard on mock data

Goal: a polished, responsive dark dashboard built against a scripted mock stream, so the whole UI is finished and reviewed before it ever touches the backend. The `modern-ui-design` skill loads automatically for work inside `web/`; every prompt below also names it

so the model does not forget. Phase 10 has three BUILD prompts with a gate after each. The checks marked BROWSER use Antigravity's browser tool and screenshots.

What you are building: a left icon rail, a top bar with a connection chip, and a Live page with the video stage and face overlays, a large Decision card, three Trust gauges (liveness, attribution, addressee), a per-face Sync timeline, and an Event log. Teal means accept, red reject, amber listening, slate idle, and nothing else uses those colours.

**10-A1. BUILD prompt: scaffold, tokens, primitives, shell**

```
Start: read AGENTS.md, docs/SPEC.md section 9 and .agents/skills/modern-ui-design/SKILL.md. Follow that skill for every decision. Phase 10 part 1. All work is inside web/.

1. Delete web/.gitkeep. Inside web/ run: npm create vite@5 . -- --template react-ts   (if it asks about a non-empty folder choose to ignore files and continue). Then run: npm install ; npm install lucide-react recharts framer-motion clsx @fontsource-variable/inter @fontsource/jetbrains-mono ; npm install -D tailwindcss@3.4 postcss autoprefixer vitest@2.
2. package.json scripts: dev (vite), build (tsc -b && vite build), typecheck (tsc -b), test (vitest run), preview (vite preview). Remove demo files: src/App.css, src/assets, the Vite logo.
3. Tailwind: write tailwind.config.js (content ['./index.html','./src/**/*.{ts,tsx}'], darkMode 'class', theme.extend.colors mapping each token to rgb(var(--token) / <alpha-value>) for: bg, elev, card, border, text, muted, accept, reject, listen, info, idle; fontFamily sans ['Inter Variable','system-ui','sans-serif'] and mono ['JetBrains Mono','ui-monospace','monospace']; borderRadius card 16px; boxShadow glass '0 8px 30px rgba(0,0,0,.35)'). Write postcss.config.js. In src/index.css add @tailwind base/components/utilities, :root with the tokens as space-separated RGB triplets (bg 10 14 20; elev 15 21 32; text 230 237 243; muted 139 152 169; accept 45 212 191; reject 248 113 113; listen 251 191 36; info 167 139 250; idle 100 116 139; card and border use white at alpha through utility classes bg-white/[0.04] and border-white/[0.08]), html and body background bg, color text, font sans, a custom thin scrollbar, :focus-visible outline 2px accent offset 2px, and a prefers-reduced-motion block that disables transitions and animations.
4. src/main.tsx imports '@fontsource-variable/inter', '@fontsource/jetbrains-mono/400.css', '@fontsource/jetbrains-mono/500.css', index.css. Add class 'dark' on html. No CDN links anywhere; the app must work offline.
5. src/types.ts: type Decision (the 8 names of SPEC section 4); interfaces FaceScores, FrameResult (fields exactly as SPEC section 4, bbox as [number,number,number,number], angles number | null, nan becomes null), HelloMsg {type:'hello', config: Record<string, Record<string, unknown>>, decisions: Decision[]}, FrameMsg {type:'frame', data: FrameResult}, UtteranceMsg {type:'utterance', data: FrameResult}, StatusMsg {type:'status', data: {running: boolean, source: string | null, error: string | null}}, type ServerMsg = union of the four.
6. src/lib/decision.ts: DECISION_META: Record<Decision, {label, short, tone: 'accept'|'reject'|'listen'|'idle', icon: LucideIcon, description}>. Labels: IDLE 'Idle', LISTENING 'Listening', ACCEPT 'Accepted', REJECT_SPOOF 'Rejected: not live', REJECT_AMBIGUOUS 'Rejected: unclear speaker', REJECT_NOT_ADDRESSED 'Rejected: not for the robot', REJECT_LOW_QUALITY
```

```
'Rejected: poor view', REJECT_NO_FACE 'Rejected: no face'. Icons from lucide-
react: Moon, AudioLines, ShieldCheck, ShieldAlert, Users, EyeOff, CameraOff,
UserX (if a name does not exist in the installed version choose the closest
lucide icon and tell me). Also export toneClasses(tone) returning Tailwind
class strings for text, bg-at-12-percent, border-at-30-percent and ring.
7. src/lib/format.ts: fmtScore(x: number | null) -> '0.72' (2 dp, '--' for
null or NaN), fmtAngle(x) -> '12°' (0 dp, '--' for null), fmtFps(x) -> '24',
fmtTime(seconds) -> 'mm:ss', clamp01(x).
8. Components in src/components/: Card.tsx (props title?, icon?, action?,
accent?: tone, className; glass style from the skill: backdrop-blur-xl, 1px
white/8 border, radius card, a soft radial glow of the accent tone at 6
percent in the top-left corner), Chip.tsx (props tone, children, icon?,
mono?), Skeleton.tsx (shimmering block, respects reduced motion),
EmptyState.tsx (icon, title, hint), ErrorState.tsx (message, onRetry),
AppShell.tsx.
   AppShell: 72 px left rail (fixed, lucide icons Activity for Live,
PlayCircle for Replay, BarChart3 for Results, Settings for Settings, each a
button with aria-label and a tooltip, the active one with a teal indicator
bar), a top bar (product name 'TrustSync' with a small mark, the page title,
ConnectionChip on the right), and below 768 px the rail becomes a bottom tab
bar with the same four buttons. src/components/ConnectionChip.tsx: props
state 'connected'|'mock'|'connecting'|'disconnected' shown as a dot plus
label (connected teal, mock violet, connecting amber with a pulse,
disconnected red).
9. src/lib/useHashRoute.ts: returns [route, navigate] for hash routes #/live
(default), #/replay, #/results, #/settings. src/App.tsx renders AppShell and,
for now, an EmptyState titled with the page name and the hint 'Built in the
next step' for every route.
10. Tests (vitest, web/src/lib/*.test.ts): format functions (incl. null, NaN,
rounding, fmtTime(75) is '01:15'); every Decision has a DECISION_META entry
with a non-empty label and an icon; toneClasses returns different strings per
tone.
11. tools/gates/gate_10.py (repo root, Python): rows (all via subprocess with
cwd=web, using shutil.which('npm'); FAIL with a clear detail if npm is
missing): npm run typecheck; npm run build; npm test; dist/index.html exists;
no string matching https?:// in src/ or index.html other than
'http://www.w3.org' and 'localhost' or '127.0.0.1' (offline rule);
tailwind.config.js mentions all 11 token names; these files exist:
src/types.ts, src/lib/decision.ts, src/lib/format.ts,
src/components/Card.tsx, Chip.tsx, Skeleton.tsx, EmptyState.tsx,
ErrorState.tsx, AppShell.tsx, ConnectionChip.tsx. A manual row (SKIP unless
hardware): prints 'BROWSER: run npm run dev in web/ and open
http://localhost:5173'.
```

```
12. Run: python tools/gates/run_gate.py 10

DO NOT install react-router, shadcn, MUI, or any component library. DO NOT
use emoji as icons. Report back in the AGENTS.md format.
```

**10-V1. BROWSER verification** (new chat, paste after U1; the model must actually open the page)

```
Follow .agents/skills/modern-ui-design/SKILL.md section Verification. In web/
run npm run dev. Use the browser tool to open http://localhost:5173 at
1440x900 and again at 390x844, taking a screenshot each time, and click each
rail or tab button. Report in exactly this form: (1) viewport, (2) what is
visible, (3) any clipped, overlapping or low-contrast element, (4) console
errors count, for both viewports. Then list 3 visual defects you would fix.
Do not change any file.
```

### 10-A2. BUILD prompt: mock stream, video stage, decision, gauges

Start: read AGENTS.md, docs/SPEC.md sections 4 and 9, the skill modern-ui-design, and web/src/types.ts. Phase 10 part 2. Work only inside web/ (and gate_10.py at the end).

1. src/lib/mock.ts: pure function mockFrame(k: number): FrameResult for frame index k at 20 Hz (t = k/20), and MOCK_CONFIG (the SPEC section 3 gate and addressee defaults: min_sync 0.35, min_margin 0.10, min_directed 0.5, min_quality 0.5). The script is a cycle of 7 acts of 7.0 s each (49 s, 980 frames). Within an act: 0 to 1.2 s decision IDLE, speech_active false; 1.2 s to 3.7 s decision LISTENING, speech_active true, sync of the speaker rising smoothly from 0.1 to a value that depends on the act; at exactly frame (act start + 3.7*20) the single frame with utterance_final true carrying the final decision; afterwards IDLE again until the act ends. Acts: 0 live accept (one face id 0 centred, final sync 0.78, directed 0.95, ACCEPT, speaker 0); 1 replay spoof (one face, sync 0.04, lip_std 0.001, REJECT_SPOOF); 2 two faces id 0 left and id 1 right, id 1 sync 0.74 and id 0 sync 0.18, ACCEPT speaker 1; 3 not addressed (yaw 45, directed 0.1, sync 0.7, REJECT_NOT_ADDRESSED); 4 no face (empty faces, REJECT_NO_FACE); 5 ambiguous (two faces both sync about 0.6, REJECT_AMBIGUOUS); 6 low quality (quality 0.2, REJECT_LOW_QUALITY). env is 0.002 during idle and 0.05 to 0.12 jittering deterministically (use a seeded sine mix, never Math.random) while speech_active. Face boxes are normalized, face sizes about 0.22 wide and 0.35 tall, and gently drift with a sine so overlays visibly move. fps field 24. Every numeric field is deterministic in k.
2. src/lib/mock.test.ts: over k in 0..979: exactly 7 frames have utterance_final, their decisions in order are ACCEPT, REJECT_SPOOF, ACCEPT, REJECT_NOT_ADDRESSED, REJECT_NO_FACE, REJECT_AMBIGUOUS, REJECT_LOW_QUALITY; all 8 Decision values appear at least once; every score field is within [0,1]; bbox values within [0,1] with x1<x2 and y1<y2; mockFrame(k) equals mockFrame(k) (deterministic) and mockFrame(980) equals mockFrame(0) except nothing else changes (cycle).
3. src/lib/useTrustStream.ts: hook useTrustStream(mock: boolean) returning {state: 'connected'|'mock'|'connecting'|'disconnected', frame: FrameResult | null, history: FrameResult[] (last 200 frames), events: FrameResult[] (utterance_final frames, newest first, max 50), config, status}. For now only mock mode is implemented: a setInterval at 50 ms advances k and sets state 'mock'. A held decision: export also held: {frame: FrameResult, until: number} | null, set when an utterance_final frame arrives and cleared 4000 ms later. The real WebSocket branch throws a clear 'not implemented in Phase 10' error that is not reached while mock is true. Mock mode is true when the URL query has mock=1 or when the env var VITE_MOCK is 1.
4. src/components/VideoStage.tsx: props {frame, mock, mirror}. Container relative, aspect-[4/3], overflow hidden, rounded-2xl, border white/8. In mock

```
mode draw an abstract camera scene: a dark vertical gradient with soft
blurred circle 'heads' at each face bbox (no photos, SVG only). In live mode
(not used yet) an img with src '/stream.mjpg' alt 'Camera stream'. On top, an
SVG with viewBox 0 0 1 1, preserveAspectRatio none, drawing per face a
corner-bracket box (non-scaling 2 px stroke, arm length 15 percent of the
side) coloured by role: speaker teal, rejected best face red, other idle
slate; label chips as absolutely positioned HTML (left and top in percent)
'ID 0 · sync 0.72' in mono, a thin sync bar under the box, and a short gaze
arrow from the box centre scaled by yaw and pitch (3 px per degree, clipped
to 100 px). When mirror is true, x coordinates become 1 - x in the overlay
code (never mirror text). Top-left pill with the decision label and icon,
top-right pill with 'FPS 24' in mono, bottom 48 px glass strip with
EnvelopeStrip. Overlays animate position with a 120 ms CSS transition.
5. src/components/EnvelopeStrip.tsx: props history; an inline SVG polyline of
the last 200 env values, normalized to the max of 0.15, stroke amber while
the latest frame is speech_active else slate, plus a faint baseline.
6. src/components/DecisionCard.tsx: props {frame, held}. Shows the held
decision (if any) else the live decision. Big icon in a 56 px rounded square
tinted with the tone, the label in 28 px semibold, the reason in muted 14 px,
a Chip 'Speaker 1' when speaker_id is not null, and 'liveness 0.78 ·
addressee 0.95' in mono. The whole card has an aria-live=polite region
announcing label changes. Decision changes animate 200 ms with framer-motion
AnimatePresence mode='wait' (scale 0.97 to 1 and opacity), and use plain CSS
when prefers-reduced-motion is set (useReducedMotion).
7. src/components/TrustGauges.tsx and Gauge.tsx: three 120 px semicircle
gauges (SVG arc, stroke 10, rounded caps, a threshold tick) for Liveness
(frame.liveness vs config gate.min_sync), Attribution (attribution_conf vs
gate.min_margin), Addressee (addressee vs gate.min_directed). Value in the
middle in 24 px mono, label under it. Arc colour: teal if value >= threshold,
red if below and the decision is a reject, amber while LISTENING and below,
slate when IDLE. The arc animates with a 300 ms stroke-dashoffset transition.
8. App.tsx: route #/live renders a temporary stack of VideoStage,
DecisionCard and TrustGauges in the 12-column grid from the skill (video 8
columns, right column 4) fed by useTrustStream(mock). Other routes unchanged.
9. tools/gates/gate_10.py: add rows for: the mock test passing is part of npm
test; files exist: mock.ts, useTrustStream.ts, VideoStage.tsx,
EnvelopeStrip.tsx, DecisionCard.tsx, TrustGauges.tsx, Gauge.tsx.
10. Run: python tools/gates/run_gate.py 10

Report back in the AGENTS.md format.
```

### 10-V2. BROWSER verification (new chat)

```
Follow the Verification section of .agents/skills/modern-ui-design/SKILL.md.
Run npm run dev in web/. Open http://localhost:5173/?mock=1 at 1440x900 and
take screenshots at about 2 s, 8 s, 15 s, 22 s, 29 s, 36 s and 43 s after
load (the mock cycles 7 acts of 7 s). For each screenshot say which decision
is shown in the Decision card and whether the video pills, face boxes and
gauge colours match it. Then repeat one screenshot at 390x844. Confirm: face
boxes move, gauges change colour with the decision, the decision text is
readable, 0 console errors. List up to 5 concrete visual defects. Do not
change any file.
```

### 10-A3. BUILD prompt: timeline, event log, final Live page

```
Start: read AGENTS.md, the skill modern-ui-design, web/src/types.ts and
web/src/lib/useTrustStream.ts. Phase 10 part 3. Work only inside web/ (and
gate_10.py).

1. src/components/SyncTimeline.tsx: recharts ResponsiveContainer + LineChart
over the last 200 history frames (10 s at 20 Hz): x axis is seconds relative
to now (-10 to 0, tick every 2 s), y axis 0 to 1 with 3 ticks; one Line per
track id found in history (monotone, 2 px, no dots), colours: the track equal
to the latest speaker_id teal, others violet, then amber, then slate, kept
stable per track id; a dashed horizontal ReferenceLine at the gate.min_sync
threshold from config with the label 'min sync 0.35'; horizontal gridlines
only at 6 percent opacity; a custom dark glass tooltip showing 'ID 1 · sync
0.74'. Speech periods are shaded behind the lines (ReferenceArea, white at 4
percent). Wrapped in Card title 'Lip-audio sync' with a legend of Chips.
Empty state 'Waiting for faces' when no data. Chart height 220 px, min width
0, no horizontal page overflow at 390 px.
2. src/components/EventLog.tsx: Card title 'Decisions' with a table (on
narrow screens stacked rows): time (fmtTime of the frame t relative to the
first event), decision Chip with the tone, speaker ('ID 1' or '-'), liveness,
addressee (fmtScore, mono), and the reason on a second muted line truncated
to 2 lines with a title attribute for the full text. Newest first, keeps 50,
a subtle 150 ms fade-in for a new row, scrolls inside the card with max-
height 320 px. Empty state 'No utterances yet. Speak a command.'.
3. src/pages/LivePage.tsx: the final layout: 12-column grid, gap 24 px:
VideoStage (8 columns) with a small toolbar above it (Mirror toggle button
with aria-pressed, a segmented control 'Overlay on/off'), right column (4
columns) with DecisionCard then TrustGauges; second row SyncTimeline (8
columns) and EventLog (4 columns). Below 1024 px the right column goes under
the video; below 768 px everything is one column in the order video,
decision, gauges, timeline, log. Add loading skeletons for the first 400 ms
of the hook (state connecting) and an ErrorState when state is
'disconnected'. App.tsx renders LivePage on #/live.
4. Polish pass following the skill: consistent 24 px page padding (16 px on
mobile), every Card has the same header height, numbers use tabular-nums,
focus rings visible on every button, rail tooltips on hover and focus, no
layout shift when decisions change (reserve space), minimum touch target 44
px on mobile tabs.
5. Add a vitest test web/src/lib/format.test.ts case for fmtTime of 0 and
3599, and web/src/components/EventLog.test.tsx only if you can do it without
adding a test-library dependency (otherwise skip it and say so).
6. tools/gates/gate_10.py: add file-exist rows for SyncTimeline.tsx,
```

```
EventLog.tsx, pages/LivePage.tsx.
7. Run: python tools/gates/run_gate.py 10

Report back in the AGENTS.md format.
```

**10-V3. Final BROWSER verification and design review** (new chat)

```
Follow the Verification section of .agents/skills/modern-ui-design/SKILL.md.
Run npm run dev in web/ and open http://localhost:5173/?mock=1 at 1440x900,
1024x768 and 390x844; take a screenshot of each after the first utterance
decision has appeared. Review as a demanding product designer. For each
viewport answer: (a) is there a clear focal point, (b) are teal, red, amber
and slate used only for state, (c) any text under 4.5:1 contrast, (d) any
element clipped or overlapping, (e) does the layout match the skill (rail,
12-column grid, card style). Press Tab through the page and report whether
every interactive element shows a visible focus ring. Report console errors.
Give a ranked list of the 5 most valuable fixes. Do not change any file.
```

Apply the fixes with one short prompt per fix (U1 first, then the single change, then `python tools/gates/run_gate.py 10`). Tag `phase-10-pass` when the gate is green and the three browser reviews are clean.

## Phase 11: connect the UI, replay mode, settings, results

Goal: the same dashboard driven by the real backend, a Replay page that runs recorded clips through the pipeline and shows expected versus actual, a Settings page with live threshold sliders, and a Results page drawn natively from `reports/results.json`. Three BUILD prompts again.

**11-A1. BUILD prompt: real stream and source control**

```
Start: read AGENTS.md, docs/SPEC.md sections 8 and 9, the skill modern-ui-design, and the files web/src/lib/useTrustStream.ts, web/src/lib/mock.ts, web/src/types.ts. Phase 11 part 1.

1. web/vite.config.ts: add server.proxy for '/api' and '/stream.mjpg' to http://127.0.0.1:8000 and '/ws' to ws://127.0.0.1:8000 with ws: true. Keep everything else.
2. src/lib/streamReducer.ts: pure function streamReducer(state: StreamState, msg: ServerMsg | {type:'tick-hold'}): StreamState where StreamState has frame, history (max 200), events (utterance frames, newest first, max 50), config (or null), status ({running, source, error} or null) and held ({frame, until} or null). Behaviour: hello sets config; frame sets frame and appends to history (trim to 200); utterance prepends to events (trim to 50) and sets held with until = now + 4000 ms (pass now through the message as an optional field ts, default Date.now()); status sets status; tick-hold clears held when Date.now() >= held.until. Move the mock mode of useTrustStream to dispatch the same message types through this reducer (frame messages from mockFrame, an utterance message when utterance_final, a hello with MOCK_CONFIG at start).
3. src/lib/useTrustStream.ts: implement the real branch with a WebSocket to ws(s)://<location.host>/ws. State machine: connecting -> connected on open -> disconnected on close/error with automatic reconnect using exponential backoff 500 ms, 1 s, 2 s, 4 s, then 5 s forever; stop retrying and close the socket on unmount. Parse each message with JSON.parse inside try/catch and ignore malformed ones (count them in a ref for debugging). Dispatch into the reducer. Keep the mock branch selecting on mock=1 or VITE_MOCK.
4. src/lib/api.ts: typed fetch wrappers getSessions(), startSource(body), stopSource(), getConfig(), setConfig(section, key, value), getResults() returning a discriminated result {ok: true, data} or {ok: false, status, message}; no thrown exceptions for HTTP errors; 8 s timeout via AbortController.
5. src/components/SourceSelector.tsx in the top bar of the Live page: a segmented control Webcam | Recorded clip, for Recorded clip a select listing sessions as 'live_single_pratik_001 · expects ACCEPT', and a Start/Stop button (Play and Square icons, aria-labels). Disabled with a tooltip in mock mode. Shows the status.error as a red Chip when present. Starting shows a loading state on the button until the status message confirms running.
6. src/components/Toast.tsx and a tiny useToast hook (context provider in App.tsx): bottom-right glass toasts with tone, auto-dismiss after 4 s, aria-live=polite.
7. VideoStage live mode: img src '/stream.mjpg'; if the image fires onError show an inline 'Stream unavailable' overlay with a Retry button that reloads the img with a cache-busting query.
```

```
8. Tests (vitest): web/src/lib/streamReducer.test.ts covering each message
type, history trimming at 200 with 250 frames, events trimming at 50, held
expiry, and that status survives later frame messages.
9. tools/gates/gate_11.py: runs gate 10's web rows again by importing
gate_10.run, and adds a row for src/lib/streamReducer.ts, api.ts,
SourceSelector.tsx, Toast.tsx existing.
10. Run: python tools/gates/run_gate.py 11

DO NOT change backend code. Report back in the AGENTS.md format.
```

### 11-A2. BUILD prompt: Replay page, demo clip, Settings page

```
Start: read AGENTS.md, the skill modern-ui-design, docs/SPEC.md sections 7
and 8, and web/src/lib/api.ts. Phase 11 part 2.

1. tools/make_demo_session.py (Python): creates data/sessions/demo_replay_000
with files and meta.json valid for tools/validate_session.py, built from
tests/fixtures/face_frontal.jpg exactly as
tests/helpers_world.make_pseudo_session does (a static real face with
replayed speech: scenario replay_spoof, expected REJECT_SPOOF, subjects
['demo']). Exits with a clear message if the fixture is missing. Idempotent:
refuses to overwrite.
2. src/pages/ReplayPage.tsx: left column (4 of 12 columns) a scrollable list
of sessions from getSessions() as selectable cards: scenario Chip, expected
Chip (tone from the expected label: ACCEPT teal, REJECT_* red, NONE slate),
subjects, id in mono; filter chips by scenario at the top; empty state 'No
recorded clips yet. Run tools/record_session.py or
tools/make_demo_session.py.'. Right column (8 columns) the same VideoStage,
DecisionCard and TrustGauges as the Live page driven by
useTrustStream(false), with a 'Run this clip' button that calls
startSource({type:'session', id}). When an utterance event arrives for the
running clip show a result banner above the stage: a green 'Matches expected'
or red 'Differs from expected' Chip comparing the decision with the clip's
expected label (ACCEPT matches ACCEPT; any REJECT_* matches REJECT_ANY; the
exact string matches a specific REJECT label; NONE matches no event). Also a
progress bar of the clip run using status.running and elapsed time.
3. src/pages/SettingsPage.tsx: loads getConfig(); sliders (custom styled
range inputs, value shown in mono, units in the label) for gate.min_sync (0
to 1, step 0.01), gate.min_margin (0 to 0.5, 0.01), gate.min_directed (0 to
1, 0.01), gate.min_quality (0 to 1, 0.01), addressee.max_yaw_deg (5 to 90,
1), addressee.max_pitch_deg (5 to 60, 1), addressee.yaw_offset_deg and
pitch_offset_deg (-30 to 30, 1), addressee.gaze_weight (0.05 to 1, 0.05), a
switch for addressee.use_gaze, and sync.min_lip_std (0.001 to 0.03, 0.001).
Each change is sent with setConfig after 400 ms of no movement; on success a
toast 'Applied. The pipeline re-calibrates for about 1 s.'; on a 422 revert
the slider and toast the server message. A 'Reload from server' button. In
mock mode show a violet Chip 'Mock: changes are not sent' and skip the calls.
Group the sliders in Cards titled Trust gate, Addressee, Sync.
4. App.tsx routes: #/replay renders ReplayPage, #/settings renders
SettingsPage. The route rail highlights correctly.
5. Tests: add a vitest test for the expected-vs-actual comparison function
(extract it as src/lib/matchExpected.ts with matchExpected(expected, decision
| null): boolean) covering all expected labels and the NONE case.
6. tools/gates/gate_11.py: add rows: tools/make_demo_session.py runs and its
session passes tools/validate_session.py (SKIP if the fixture is missing);
```

```
file-exists rows for ReplayPage.tsx, SettingsPage.tsx, matchExpected.ts.
7. Run: python tools/gates/run_gate.py 11

Report back in the AGENTS.md format.
```

### 11-A3. BUILD prompt: Results page

Start: read AGENTS.md, the skill modern-ui-design, docs/SPEC.md and the 8-A3 description of reports/results.json (keys: generated_at, dataset, config, current, calibrated, loso_rule, loso_learned, learned_kind, ablation, av_lag, fps, figures, notes; each metrics dict has decision_accuracy, far_spoof, far_all_reject, frr, false_trigger_rate, missed_utterance_rate, liveness{precision,recall,f1}, attribution_accuracy, addressee_accuracy, reason_accuracy, per_scenario{scenario:{n,accuracy,accept_rate}}, confusion{expected:{decision:count}}). Phase 11 part 3.

1. web/public/mock-results.json: a small clearly fictional sample matching that schema (6 scenarios, 3 subjects, plausible but obviously sample numbers, notes: ['Sample data for the mock UI']). Never present it as real.
2. src/lib/results.ts: type ResultsFile; headline(results) picks loso_learned.pooled if present, else loso_rule.pooled, else current; formatPct(x) -> '91.7%'; scenarioRows(metrics) sorted by scenario name. Vitest tests for headline selection order and formatPct (0, 1, 0.9166).
3. src/pages/ResultsPage.tsx: loads getResults() (in mock mode fetch /mock-results.json). States: skeletons while loading, EmptyState 'No evaluation yet. Run python -m trustsync.eval.run_eval.' on 404, ErrorState with Retry on other errors. When mock data is shown, a violet 'Sample data' Chip in the page header. Layout: a KPI row of 6 Cards (Decision accuracy, Spoof false-accept rate with teal when 0 else red, Liveness F1, Attribution accuracy, Addressee accuracy, Pipeline FPS or '--'), each with a big mono value and a caption naming what it measures; below, left: Ablation Card (recharts grouped BarChart: for each variant B0 to A5, bars for accuracy (teal) and false-accept rate (red), y 0 to 1, a caption explaining B0 is the naive voice interface); right: Confusion Card (CSS grid heatmap, rows expected, columns decision short labels, cell intensity by count, numbers in mono, a legend); next row left: Per-scenario Card (horizontal bars of accuracy with n in the label); right: Leave-one-subject-out Card (table of subject, accuracy, far_all_reject, plus the pooled row); last row: Dataset Card (n_clips, counts per scenario as Chips, subjects) and Notes Card (the notes list and generated_at, plus av_lag median and saturated_fraction if present). Every chart has an accessible text summary in a visually hidden element.
4. App.tsx: #/results renders ResultsPage.
5. tools/gates/gate_11.py: add file-exist rows for ResultsPage.tsx, results.ts, public/mock-results.json (valid JSON with all keys). Add the integration row: build the web app, start uvicorn (python -m trustsync.server.app --port <free port>) as a subprocess with cwd at the repo root, then: GET / returns HTML containing ' '; GET /api/sessions returns a list; if data/sessions/demo_replay_000 exists, POST /api/source session demo_replay_000 and read the WebSocket until an utterance message arrives (timeout 30 s) whose decision is REJECT_SPOOF (SKIP if the

```
demo session is missing); GET /api/results returns 200 or 404; terminate the
subprocess in a finally block.
6. Run: python tools/gates/run_gate.py 11

Report back in the AGENTS.md format.
```

**11-V. BROWSER verification of the integrated app** (new chat; needs `python tools/make_demo_session.py` and `npm run build` done)

```
Run python -m trustsync.server.app in the background. Using the browser tool
open http://127.0.0.1:8000 (the built app is served by the backend) at
1440x900. Do these steps and screenshot after each: (1) the Live page shows
connected state; (2) open Replay, select demo_replay_000, press Run this
clip, wait 12 s: the Decision card must show Rejected: not live and the
banner must say Matches expected; (3) open Settings, move min sync to 0.9,
wait 2 s, confirm the toast, run the clip again and note the decision; (4)
open Results and report whether it shows the empty state or real numbers; (5)
stop the server and confirm the UI shows disconnected within 3 s, restart the
server and confirm it reconnects without a page reload. Report console errors
per step. Do not change any file.
```

**11-C. VERIFY checks** (U5 with N=11)

```
1. Run npm run typecheck, npm run build and npm test in web/ and paste the
summaries.
2. Run the gate-11 integration row by hand and paste its output.
3. Kill the backend while the Live page is open (browser tool) and confirm
the reconnect backoff by reading the network log for the WebSocket attempts:
intervals roughly 0.5, 1, 2, 4, 5 seconds.
4. Grep web/src for 'Math.random' and report matches in non-test files (there
must be none in mock.ts).
5. Grep the built dist/ for external hostnames (anything starting http but
not localhost or 127.0.0.1 or w3.org). There must be none.
```

# Phase 12: robustness and benchmark

Goal: prove the Evaluation Plan's last two test cases (partial occlusion or low light must degrade gracefully, never falsely accept) and its last metric (end-to-end FPS and latency), and harden the pipeline against the dull failures that kill live demos: odd frames, flickering faces, clock jumps, a long run.

**12-A1. BUILD prompt: hardening and stress tests**

```
Start: read AGENTS.md, docs/SPEC.md, src/trustsync/pipeline.py and
tests/helpers_world.py. Phase 12 part 1.

1. Hardening in src/trustsync/pipeline.py (the only source change in this
prompt, plus the one in step 2):
   - process(frame_bgr, t) raises ValueError('frame must be HxWx3 uint8')
unless frame_bgr is a numpy array with ndim 3, 3 channels, dtype uint8 and
both sides at least 1; raises ValueError('t must be finite') if t is NaN or
infinite.
   - If t is more than 1e-6 smaller than the previous call's t (clock went
backwards), log one warning, clear all per-track buffers, the utterance
tracker and the VAD, and continue (no exception).
   - A face whose pts contain NaN or whose bbox is not finite is dropped for
that frame (logged at debug level).
2. In src/trustsync/server/runtime.py (additive): the worker loop catches an
exception from a single frame, logs it, counts consecutive failures, and only
after 30 consecutive failures stops and reports the last error in status. A
successful frame resets the counter. Existing tests must still pass.
3. tests/test_robustness.py using tests/helpers_world.py (add parameters to
build_world if needed, without changing existing behaviour):
   a. Frame fuzz through the real FaceTracker: all-black, all-white, uniform
noise, an 8x8 frame, a 1x1 frame: no exception, no face (or a face with
quality below 0.5), decision never ACCEPT. A 2-D grayscale frame, a uint16
frame and a 4-channel frame each raise ValueError. NaN t raises ValueError.
   b. Face flicker: live_single world where the face list is empty on a
random 20 percent of frames (rng per seed): over 30 seeds at least 90 percent
end ACCEPT; never ACCEPT with a speaker other than 0.
   c. Track id switch: live_single world where the face id changes from 0 to
1 at the middle of the utterance: the final decision is never ACCEPT with a
wrong speaker and is a REJECT_*; record which one in the test output.
   d. Two faces both driven by the same speech (matched, lag 0.033):
REJECT_AMBIGUOUS in at least 90 percent of 20 seeds, never ACCEPT.
   e. Loud audio: the live_single world with env multiplied by 20 gives the
same final decision as the normal one in 20 seeds.
   f. Noisy audio: baseline noise raised from 0.002 to 0.02 (speech peaks
unchanged): print the share of seeds that end ACCEPT over 30 seeds
(informational) and assert it is at least 0.5.
   g. Clock jump: at frame 120 shift all later t by -5.0 s: no exception, and
a later utterance (add a second burst after the jump) is still decided.
   h. Mouth covered, simulated: live_single world where the single face has
quality 1 but the lip signal is static for the whole run: REJECT_SPOOF (never
ACCEPT).
   i. Darkness curve: for brightness values 5, 15, 30, 45, 60, 90 build a
```

```
flat grey frame with a drawn bright ellipse; face_quality is non-decreasing
with brightness.
   j. Long run (mark with @pytest.mark.slow and register the marker in
pyproject): 18000 frames (10 minutes at 30 fps) of the live_single world
looped with bursts every 6 s: memory growth below 100 MB, no exception, per-
frame time p99 below 5 times p50, and exactly one utterance_final per burst.
   Register the 'slow' marker in pyproject.toml. In tests/conftest.py add a -
--runslow command-line option (pytest_addoption) and a
pytest_collection_modifyitems hook that skips every test marked slow unless -
--runslow is given. In tools/gates/run_gate.py the default pytest row stays
unchanged (it therefore skips slow tests), and when the flag --slow is passed
add a second row that runs pytest -q -m 'not hardware' --runslow
tests/test_robustness.py.
4. tools/gates/gate_12.py: rows for tests/test_robustness.py; a hardware row
(SKIP unless hardware) for the two live checks: 'Cover your mouth completely
with your hand and say a command' then 'Turn the room lights low or cover
half the camera lens and say a command'; for each, run the pipeline for 8 s
and PASS only if no utterance_final decision is ACCEPT (no utterance at all
is also a PASS).
5. Run: python tools/gates/run_gate.py 12 and then python
tools/gates/run_gate.py 12 --slow

DO NOT lower any limit in the tests. Report back in the AGENTS.md format.
```

### 12-A2. BUILD prompt: benchmark

```
Start: read AGENTS.md, docs/SPEC.md and src/trustsync/pipeline.py. Phase 12
part 2. Create exactly:

1. tools/benchmark.py. Arguments: --source webcam|photo|session (default
photo), --session PATH, --frames N (default 300), --out
reports/benchmark.json. Sources: 'photo' repeats
tests/fixtures/face_frontal.jpg N times with a synthetic audio source (white
noise shaped by synth.make_env with a burst every 3 s) and t advancing by
1/30; 'session' reads the session video and audio; 'webcam' reads the live
camera and microphone for N frames. Measure with time.perf_counter, after 30
warm-up frames that are excluded: per-stage time for face_tracker.process,
pose+gaze+directed for all faces, running sync for all faces, the full
pipeline.process, JPEG encode of the frame at the configured quality, and
draw_hud. Print a table with columns stage, mean ms, p50, p95, p99. Compute
throughput FPS = 1000 / mean of the full pipeline.process time (and p5 FPS
from p95 time). Measure decision latency on the photo or synthetic source:
for each utterance, seconds between the true end of the burst and the t of
the utterance_final frame (report mean and max). Write reports/benchmark.json
with keys machine {platform, processor, python, cpu_count, opencv,
mediapipe}, source, n_frames, stages {name: {mean_ms,p50_ms,p95_ms,p99_ms}},
fps {mean, p5}, decision_latency_s {mean, max}. If reports/results.json
exists, also set its 'fps' key to {mean, p5, source, machine: processor}
without touching any other key (read, modify, write atomically via a temp
file and rename).
2. tests/test_benchmark_logic.py: factor the percentile and table code into
functions in tools/benchmark.py (or a module it imports) and test them on
fixed arrays (p50, p95, p99 of 1..100; FPS from mean ms 25 is 40.0); test
that the results.json update only changes the 'fps' key (use a tmp copy).
3. tools/gates/gate_12.py: add a row 'benchmark photo run' that runs python
tools/benchmark.py --source photo --frames 150 (SKIP if the fixture is
missing) and checks reports/benchmark.json has all keys; add two GOAL rows
reading it: fps.mean >= 15 and decision_latency_s.max <= 0.8 (detail prints
the numbers; if the FPS goal fails the detail suggests setting
video.max_faces to 2 and video.width and height to 480x360 in config.yaml and
re-running).
4. Run: python tools/gates/run_gate.py 12

Report back in the AGENTS.md format.
```

### 12-B. You run the live checks and the real benchmark

```
python tools/gates/run_gate.py 12 --hardware
python tools/benchmark.py --source webcam --frames 600
```

**12-C. VERIFY checks** (U5 with N=12)

```
1. Run python tools/gates/run_gate.py 12 --slow and paste the table.
2. Re-run tests/test_robustness.py with three different seeds by editing only
the seed constant in a temporary copy (not committed) and report whether any
case flips.
3. Print reports/benchmark.json and confirm every stage time is positive and
stage means sum to less than the full pipeline mean plus 20 percent (stages
are measured separately).
4. Confirm in pipeline.py that none of the new validation raises inside the
lock or leaves state half-updated: describe what state is unchanged after a
ValueError.
```

## Phase 13: documentation, demo kit, release

Goal: everything your Deliverables section promises that is not code: README, architecture notes, report assets generated from real results (so no number is ever typed by hand), a demo script for the four demonstration scenarios, and a clean-clone check.

**13-A1. BUILD prompt: docs and report assets**

Start: read AGENTS.md, docs/SPEC.md and reports/results.json (if it does not exist, run python -m trustsync.eval.run_eval --records-json <a synthetic records file made by a throwaway script> --out reports/sample to get one for layout work, and mark all outputs as SAMPLE). Phase 13 part 1. Create exactly:

1. README.md: title, one-paragraph problem statement (live? who? for me?), a quick-start (Python 3.11, install order: pip install mediapipe==0.10.14 then pip install -r requirements.txt then pip install -e ., then python tools/gates/run_gate.py 6 for a smoke test), how to run the live demo (python tools/run_demo.py), how to record data (link to docs/RECORDING.md), how to evaluate (python -m trustsync.eval.run_eval --calibrate --learned), the repository layout (copy from SPEC section 2, shortened), and a table of the phases and tags. No claims of accuracy numbers in the README.
2. docs/ARCHITECTURE.md: the pipeline as an ASCII block diagram that matches the proposal's diagram (camera and microphone to landmarks, pose, envelope, per-face sync, addressee, trust gate, downstream), then one subsection per module explaining the algorithm in plain words with the formulas of SPEC section 5 (mouth opening ratio, envelope RMS, sync score with lag search, directed score, rule order of section 6) and the config keys each uses.
3. docs/RECORDING.md: the recording protocol table and example commands from the Phase 7 section of the build pack, plus the 10 prompt commands and the consent note: all recorded people gave written consent, data stays local, and faces are not published.
4. tools/make_report_assets.py: reads reports/results.json (argument --results, default reports/results.json) and writes into report/: figures/ (copy of reports/figures/*.png), tables.md (markdown tables: headline metrics, ablation B0 to A5, per-scenario accuracy with n, confusion matrix, leave-one-subject-out folds, dataset counts), tables.csv files for each table, and numbers.json (a flat dict of every number a report sentence might quote, with its source key). Every number is read from results.json, never typed. Metrics shown with the count next to the percentage (for example 22/24, 91.7 percent) and, for each proportion, a 95 percent Wilson interval computed in a function with a unit test.
5. tools/make_report_skeleton.py: writes report/REPORT.md with the section headings Abstract, Problem and motivation, Background and gap, Method (architecture, sync score, addressee, trust gate), Implementation, Dataset and protocol, Evaluation, Results, Ablation, Limitations, Conclusion, References; the Results and Ablation sections are filled by inserting the tables from report/tables.md and sentences generated from report/numbers.json using f-strings (for example: f'The full rule gate accepted {k} of {n} replay clips'), so numbers cannot drift. The other sections contain a short bullet list of the facts to write about, taken from docs/ARCHITECTURE.md, and the

```
marker TODO(student) where prose is needed. Also write
report/SLIDES_OUTLINE.md: 12 slides with a title and 3 bullets each and which
figure goes on it, and report/DEMO_SHOTLIST.md: a shot list for a 3 minute
demo video covering the four demonstrations of the proposal (live accept,
spoof reject, multi-speaker attribution, addressee rejection) with what is on
screen and what is said.
6. report/limitations.md containing the limitations list from the build
pack's 'Known limits' section as a starting point (the human will edit it).
7. tools/run_demo.py: if web/dist is missing, run npm ci and npm run build in
web/; start the server (python -m trustsync.server.app) as a subprocess, wait
until GET /api/health answers, open http://127.0.0.1:8000 with
webbrowser.open, print 'Demo running. Press Ctrl+C to stop.' and terminate
the server on Ctrl+C.
8. tests/test_report_assets.py: with a synthetic results.json,
make_report_assets writes tables.md, numbers.json and CSVs; every number in
numbers.json matches the results.json source; wilson(22, 24) is within 0.01
of (0.742, 0.968); make_report_skeleton output contains no digits that are
absent from numbers.json in the Results section (check every number token
against numbers.json, ignoring section numbers).
9. tools/gates/gate_13.py: rows for the new test file; file-exist rows for
README.md, docs/ARCHITECTURE.md, docs/RECORDING.md, report/limitations.md; a
row that README.md mentions every command it should (the install order and
run_demo); the slow row 'clean clone': git clone the repo to a temp
directory, create a venv there, install in the documented order, run pytest -
q (default set) there, PASS if exit 0 (only when --slow is passed, otherwise
SKIP with a note).
10. Run: python tools/gates/run_gate.py 13 and then python
tools/gates/run_gate.py 13 --slow

DO NOT write prose results by hand. Report back in the AGENTS.md format.
```

### 13-B. You finish the project

```
python -m trustsync.eval.run_eval --calibrate --learned --write-config
python tools/benchmark.py --source webcam --frames 600
python tools/make_report_assets.py
python tools/make_report_skeleton.py
python tools/run_demo.py
```

Then record the demo video following `report/DEMO_SHOTLIST.md`, write the `TODO(student)` prose yourself or paste `report/REPORT.md` and `report/tables.md` into a Claude chat and ask for a first draft of those paragraphs only (the numbers are already

fixed by the tables). Finally run `python tools/gates/run_gate.py 13 --hardware --slow`.

**13-V. FINAL VERIFY checks** (U5 with N=13; run on a different machine or a fresh folder if you can)

```
1. git clone the repository into a new empty folder, follow README.md exactly
as written (no extra steps from memory), and report every point where the
README was wrong or incomplete.
2. Run python tools/gates/run_gate.py 13 --slow and paste the full table,
then python tools/gates/run_gate.py 12 --hardware if a camera is available.
3. Pick 5 numbers at random from report/REPORT.md and trace each to
reports/results.json by key; report any mismatch.
4. List every TODO and FIXME in the repository.
5. Confirm the four proposal demonstrations all work in the dashboard: live
accept, replay reject, two-person attribution, not-addressed reject. Say
which of them you could not test without hardware.
```

## Known limits (state these in the report)

Stating limits yourself is what makes the evaluation credible, and a reviewer will find the first two anyway.

1. **Video replay defeats lip-sync liveness.** If an attacker plays a video of you speaking, the lips move in sync with the audio and the score is high. Your system defeats the audio-only replay in your proposal (phone speaker beside a still mouth), not a screen replay. Mitigations to name as future work: a challenge-response (the robot asks for a random word), depth or infrared sensing.
2. **Mimicry.** Someone who silently mouths along while a phone plays the audio can raise the score. The static-mouth check blocks the lazy attack, not a determined one.
3. **Correlation has a chance floor.** A mismatched but moving mouth can correlate at 0.3 to 0.5 over a 2 s window. This is why Phase 5 prints the null distribution and Phase 8 calibrates on real clips.
4. **One microphone gives no direction.** The audio envelope is global; if two people speak at once the result is REJECT_AMBIGUOUS by design.
5. **Head pose comes from a generic 3D face model.** Expect roughly 10 degrees of error; `calibrate_pose.py` removes the constant part. The camera also stands in for the robot's eyes, so mount it where the robot's gaze would be.

6. **Small dataset.** Three subjects and 64 to 108 clips give wide confidence intervals; report counts next to percentages (the Phase 13 tool adds Wilson intervals) and use leave-one-subject-out numbers as the headline.
7. **An off-camera live speaker is rejected.** Speech that cannot be matched to a visible mouth is treated as unverifiable. That is the safe choice for this application but it is a design decision to mention.
8. **Latency.** The decision arrives at the end of the utterance plus the 0.3 s VAD hangover, so it suits discrete commands, not barge-in.
9. **Privacy.** All processing is local; the dataset contains faces, so record with written consent and do not publish clips.

**Assumptions I made that you should check**

- I could not verify what the "3.8 Flash" model can do, so every prompt assumes a fast model with a small working memory: one deliverable per prompt, exact signatures, tests written into the prompt. If a prompt is too long for it, split at a numbered step as noted in Phase 6 and run the gate between halves.
- The Python version is pinned to 3.11 and MediaPipe to 0.10.14 so the `mp.solutions.face_mesh` API in the spec exists. If your machine cannot install that pair, stop at Phase 0 and tell me before changing it.
- Threshold defaults in `config.yaml` are starting guesses. Only Phase 8's calibration on your recordings should be trusted.